

AGRIGUARD: AI-POWERED CROP DISEASE DETECTION & IOT-BASED SMART IRRIGATION SYSTEM

BY

Arslan Ahmed Naseem

Registration No. 2022-GCUF-02591

Amna Ikram

Registration No. 2022-GCUF-02602

Project submitted in partial fulfillment of the requirements for the degree of

Bachelors of Science

In

Software Engineering



DEPARTMENT OF SOFTWARE ENGINEERING
GOVERNMENT COLLEGE UNIVERSITY, FAISALABAD.

June 2026

DECLARATION

This work reported in this final project was carried out by us under the supervision of Dr. Zubair Nabi, Department of Software Engineering, Government College University, Faisalabad, Pakistan.

We hereby declare that the title of project is "AgriGuard: AI-Powered Crop Disease Detection & IoT-Based Smart Irrigation System" and the contents of this project are the product of our own research, and no part has been copied from any published source (except the references, standard mathematical models/equations/formulas/protocols etc.). We further declare that this work has not been submitted for award of any other degree/diploma. The University may take action if the information provided is found inaccurate at any stage.

Signature of the Students/Scholars

Name: Arslan Ahmed Naseem

Registration No.: 2022-GCUF-02591

Name: Amna Ikram

Registration No.: 2022-GCUF-02602

DEDICATION

We dedicate this project to our parents, whose sacrifices made this journey possible.

To our mothers, who gave everything they had, quietly and without complaint, so their children could have opportunities they never got themselves. Your love carried us further than you know.

To our fathers, who loved us in the quietest ways, through early mornings, long working days, and prayers they never said out loud. You never needed words to show us you cared.

To our families, who stood by us even when they did not fully understand what we were building or why it was taking so long.

To the nights we wanted to quit, the mornings we showed up anyway, and every difficult day in between that quietly made us stronger. This project is proof that we did not give up.

And to the farmers of Pakistan, the people who wake before dawn and labour under the open sky to feed a nation that rarely stops to thank them. You are the reason this project exists. May this small effort make their burden a little lighter.

CERTIFICATE BY SUPERVISORY COMMITTEE

We certify that the contents and form of the project submitted by Arslan Ahmed Naseem, Registration No. 2022-GCUF-02591 and Amna Ikram, Registration No. 2022-GCUF-02602 has been found satisfactory and in accordance with the prescribed format. We recommend it to be processed for the evaluation by the External Examiner for the award of degree.

Supervisor

Signature of Supervisor

Name:

Designation with Stamp.....

Chairperson

Signature with Stamp.....

ACKNOWLEDGEMENTS

We wish to express our sincere gratitude to everyone who contributed to this project and to the journey that led to it.

First and foremost, all praise to **Allah Almighty**, who gave us the strength, patience, and ability to undertake this project and see it through to the end. Without His blessings, none of this would have been possible.

We are grateful to our supervisor, **Dr. Zubair Nabi**, for his guidance throughout this project. His feedback at key milestones helped us shape the direction of our work and pushed us to think more carefully about what we were building.

Our families deserve more thanks than this page can hold. Their prayers and sacrifices carried us through every step of this journey.

Arslan Ahmed: To myself, for getting through four years that were heavier than most people around me knew. There were moments I nearly did not make it to this page. I am glad I stayed.

To my mother, who gave up things she never told me about so I could be here & have a future she never got to choose for herself. I did not understand the weight of that for a long time. I do now. Everything I build from here is yours first.

To my father, who did more for us than we were ever mature enough to see. It took hitting reality to understand how much our parents quietly carry. I am sorry it took me this long and I am grateful beyond what words can hold.

To my sisters (Amna and Ayesha), who I love more than I say out loud, and who were always there without ever needing to be asked. To my brother (Rehan), for being someone I could lean on.

To Maliha Hiader, who came into my life when I least expected it and quietly changed everything, some people arrive without warning and leave you wondering how you ever managed without them. She brought light into some of my darkest days, and the person I am today owes something to her presence.

To Amna Ikram, my friend and teammate, who brought colour to this journey and made the hard days feel lighter. Some people come into your life and change the atmosphere around them. She is one of them.

Amna Ikram: To myself, for staying even when walking away felt easier. These four years tested me in ways I did not expect. I finished anyway.

To my father, who loves me in a way that always made me feel like I could do anything. Your belief in me was never quiet and I needed that more than you know.

To my mother, whose quiet strength held everything together. I was too young to see it then. I see it now.

To my sister (Tayyiba) and brothers (Zayan, Sabeeh and Abdullah), who drove me crazy and made me laugh in equal measure, and who always made coming home worth it.

To Arslan Ahmed, my friend and teammate, who was there for me not just through this project but through every difficult moment of this degree. He guided me, taught me, and never made me feel like a burden. Thanks for being there.

Signature of the Students/Scholars

Arslan Ahmed Naseem

Registration No.: 2022-GCUF-02591

Amna Ikram

Registration No.: 2022-GCUF-02602

TABLE OF CONTENTS

1	Introduction of AgriGuard	2
1.1	Mobile Technology (React Native & Expo)	2
1.2	Hardware Requirements	2
1.3	Machine Learning Technology	4
1.4	Purpose of Using Deep Learning for Crop Disease Detection	4
1.5	Importance for Pakistani Agriculture.....	5
1.6	Functionality of AgriGuard	5
1.7	Modules	6
1.8	Benefits	7
1.9	Objectives	7
1.10	Features	8
1.10.1	Features of the Mobile Application	8
1.10.2	Features of the IoT Hardware System	8
1.11	Functionality	8
1.12	Purposes & Benefits.....	8
1.13	Introduction to the Machine Learning Pipeline	9
1.14	Introduction to the Operating Environment	9
1.15	Tools Used to Develop AgriGuard	10
1.16	IoT Hardware Components.....	10
2	Software Requirements Specifications	12
2.1	Introduction of the Requirements	12
2.2	Software Requirement Specification (SRS)	12
2.3	Goals of Software Requirement Specification.....	12
2.4	Gathering Requirements	13
2.4.1	Requirements Gathering Process	13
2.5	Capturing Better Requirements.....	13

2.6 Different Types of Software Requirements	14
2.6.1 Functional Requirements	14
2.6.2 Non-Functional Requirements	16
2.8 Deployment Requirements	16
3 Software Design Description	18
3.1 Methodology	18
3.2 Unified Modeling Language (UML)	18
3.3 Designing Technique	18
3.4.1 Why We Used Agile Development	19
3.4.2 Development Phases	19
3.5 System Features	20
3.5.1 Disease Detection System.....	20
3.5.2 Smart Water Management System	22
3.5.3 Pesticide Advisory System	22
3.6 Use Case Diagram	22
3.6.1 Use Case: Farmer Detects Disease	23
3.6.2 Use Case: Farmer Controls Irrigation.....	24
3.6.3 Use Case: Farmer Manages Account.....	24
3.7 Entity Relationship Diagram	25
3.7.2 Database Schema Description	25
3.8 System Sequence Diagrams.....	27
3.8.3 Authentication Sequence	27
3.8.4 Disease Detection Sequence	29
3.8.5 Water Management Sequence.....	30
3.9 System Architecture Diagram.....	32
3.9.1 Communication Patterns.....	32
3.10 ML Pipeline Diagram	33

3.11 Hardware Communication Diagram	35
3.11.1 IoT Data Flow	35
4 Software Development & Implementation	38
4.1 User Guidelines for AgriGuard.....	38
4.1.1 Splash Screen	38
4.1.2 Login & Registration Screen	39
4.1.3 Home Screen (Dashboard)	40
4.1.4 Detect Screen	41
4.1.5 Result Screen	43
4.1.6 History Screen.....	44
4.1.7 Water Management Screen	45
4.1.8 Profile Screen and Edit Profile Screen	46
4.1.9 Supported Crops Catalog	46
4.2 Backend API Implementation.....	47
4.2.1 Server Entry Point (server.js).....	48
4.2.2 Database Connection (config/db.js).....	48
4.2.3 Brevo Email Service (config/email.js).....	48
4.2.4 JWT Authentication Middleware (middleware/authMiddleware.js)	49
4.2.5 Authentication Controller (controllers/authController.js)	50
4.2.6 Scan Controller (controllers/scanController.js)	52
4.2.7 Pesticide Controller (controllers/pesticideController.js)	53
4.2.8 ESP Device Controller (controllers/espController.js).....	54
4.2.9 ESP Service Module (services/espService.js)	56
4.3 Database Model Implementation	57
4.3.1 User Model (models/User.js)	57
4.3.2 Scan Model (models/Scan.js)	58
4.3.3 Pesticide Model (models/Pesticide.js)	59

4.3.4 OTP Model (models/OTP.js)	60
4.3.5 EspDevice Model (models/EspDevice.js)	60
4.4 ML Inference Server Implementation.....	61
4.4.1 Complete Flask Server (flask_api/app.py).....	61
4.5 ESP32 Hardware Firmware	63
4.5.1 ESP32-CAM Firmware (esp32_cam.ino).....	63
4.5.2 ESP32-WROOM Pump Controller Firmware (pump_controller.ino).....	65
4.6 Mobile App Service Layer.....	66
4.6.1 Central Configuration (config.js).....	66
4.6.2 API Service (services/api.js).....	67
4.6.3 Auth Service (services/authService.js)	67
4.7.1 Cloud Deployment Summary	69
4.7.2 Deployment Rationale	69
5 Software Test Documentation	71
5.1 Project Testing	71
5.2 Purposes of Testing.....	71
5.3 Benefits of Software Testing	71
5.3.1 Efficient Testing	71
5.3.2 Reusability	71
5.3.3 Consistency	72
5.3.4 Reliability.....	72
5.4 Testing Levels Applied.....	72
5.5 User Acceptance Testing	72
5.5.1 Who Performs User Acceptance Testing.....	73
5.6 Prerequisites of User Acceptance Testing	73
5.7 System Testing.....	73
5.7.1 Method.....	73

5.7.2 Tasks	74
5.7.3 When is it Performed	75
5.7.4 Who Performs It.....	75
5.8 Unit Testing	75
5.8.1 Who Performs It.....	76
5.9 Integration Testing	76
5.10.1 Model 1: Crop Classifier Evaluation	77
5.10.2 Model 2: Disease Diagnoser Evaluation.....	78
5.11 Acceptance Testing.....	79
5.12.1 Registration Validation	80
5.12.2 Login Validation	80
5.12.3 Disease Detection Validation.....	80
5.12.4 Water Management Validation	80
5.13 Implementation	81
5.14 System Verification	81
5.15 Alpha Testing.....	82
5.16 Beta Testing & Methodologies	82
6 Post Mortem Report.....	84
6.1 Roles & Tasks	84
6.2 What Worked Well	84
6.3 Team Effectiveness	85
6.4 Lessons Learned	85
6.5 Conclusion of the Project.....	86

LIST OF TABLES

Table 1: Hardware Component Specifications	2
Table 2: Software Tools and Versions	3
Table 3: Core System Modules Overview	6
Table 4: Functional Requirements - User Authentication	14
Table 5: Functional Requirements - Dashboard & Profile	14
Table 6: Functional Requirements - Disease Detection.....	14
Table 7: Functional Requirements - Irrigation Control	15
Table 8: Functional Requirements - Crop Catalog & History	15
Table 9: Functional Requirements - Offline Sync	15
Table 10: Non-Functional Requirements Implementation	16
Table 11: Project Timeline and Deliverables.....	19
Table 12: Sequence Diagram - Authentication Flow.....	23
Table 13: Sequence Diagram - Disease Scan Flow	24
Table 14: Sequence Diagram - Hardware Sync Flow.....	24
Table 15: Software Design Patterns.....	32
Table 16: External Service Dependencies	69
Table 17: Testing Strategy and Scope	72
Table 18: Core Functional Test Cases	74
Table 19: System Integration Testing.....	76
Table 20: User Acceptance Criteria.....	79
Table 21: Validation Rules - User Input.....	80
Table 22: Validation Rules - Device Data	80
Table 23: Validation Rules - API Requests	80
Table 24: System Behavior Validation.....	80
Table 25: Team Roles and Responsibilities.....	84

LIST OF FIGURES

Fig. 1: Activity Diagram - Disease Detection Flow	21
Fig. 2: Use Case Diagram	23
Fig. 3: Entity Relationship Diagram	26
Fig. 4: Sequence Diagram - Authentication Flows	28
Fig. 5: Sequence Diagram - Disease Detection Data Flow	29
Fig. 6: Sequence Diagram - Smart Water Management Flow	31
Fig. 7: System Architecture Diagram	33
Fig. 8: ML Pipeline Diagram Dual-Stage Inference Flow	34
Fig. 9: Hardware Communication Diagram	36
Fig. 10: Splash Screen	38
Fig. 11: Login Screen Fig. 12: Registration Screen	39
Fig. 13: Home Screen	40
Fig. 14: Detect Screen (Empty State) Fig. 15: Detect Screen (Image Selected)	41
Fig. 16: Detect Screen (Analyzing)	42
Fig. 17: Result Screen (Healthy Plant) Figure 18: Result Screen (Diseased Plant) ...	43
Fig. 19: History Screen Figure 20: History Delete Confirmation	44
Fig. 21: Water Management Screen	45
Fig. 22: Profile Screen Fig. 23: Edit Profile Screen	46
Fig. 24: Supported Crops Catalog	47
Fig. 25: Model 1 Confusion Matrix	77
Fig. 26: Model 1 Training/Validation Curves	78
Fig. 27: Model 2 Confusion Matrix	78
Fig. 28: Model 2 Training/Validation Curves	79

Chapter 1

Introduction of AgriGuard

1 Introduction of AgriGuard

Agriculture is vital to Pakistan's economy yet crop diseases and inefficient irrigation cause severe losses. AgriGuard addresses these challenges as an end-to-end smart farming ecosystem.

1. **AI-powered crop disease detection** through a dual-stage Convolutional Neural Network (CNN) pipeline built on the EfficientNetB3 architecture, capable of identifying 76 disease conditions across 10 major crops.
2. **IoT-based automated irrigation** using ESP32 microcontrollers equipped with soil moisture sensors, enabling both manual remote control and threshold-based automatic watering.

1.1 Mobile Technology (React Native & Expo)

The AgriGuard mobile application is developed utilizing React Native 0.81.5 and Expo SDK 54, a powerful framework that compiles a single JavaScript codebase into high-performance, native UI components for both Android and iOS devices. By leveraging the Expo managed workflow, the project significantly reduced development time by integrating robust, pre-built modules for camera access, secure credential storage, client-side image manipulation, and haptic feedback without requiring complex manual native module linking. Furthermore, the application employs strict component-based architecture, utilizing React Navigation for seamless screen routing, the React Context API for global authentication state management, and both Axios and the native Fetch API for secure HTTP communication with all backend microservices.

1.2 Hardware Requirements

The AgriGuard system requires the following hardware components for its IoT subsystem:

Table 1: Hardware Component Specifications

Component	Specification	Purpose
ESP32-CAM (AI-Thinker)	Dual core 240MHz, 4MB Flash, 520KB SRAM, PSRAM, built-in camera	Field-based leaf image capture

Component	Specification	Purpose
ESP32-WROOM (Dev Module)	Dual core 240MHz, 4MB Flash, 520KB SRAM, WiFi 802.11 b/g/n	Pump control and soil moisture monitoring
Capacitive Soil Moisture Sensor	Analog output, corrosion-resistant	Real-time soil moisture measurement
5V DC Water Pump	Submersible, 120L/H flow rate	Irrigation actuator
Single-channel Relay Module	5V trigger, 10A rated	Electrical isolation between ESP32 and pump
FTDI USB-to-Serial Programmer	3.3V logic	Firmware upload to ESP32-CAM (no built-in USB)
Breadboard & Jumper Wires	Standard	Prototyping connections
5V USB Power Supply	2A minimum	Powering ESP32 modules

For mobile usage, the system requires any Android (5.0+) or iOS (13.0+) smartphone with a camera and internet connectivity.

For the software development environment, the following are required:

Table 2: Software Tools and Versions

Software	Version	Purpose
Node.js	18+ LTS	Backend runtime
Python	3.9+	ML inference server
Arduino IDE	2.3.8+	ESP32 firmware development
MongoDB	6.0+	Database server (local) or MongoDB Atlas (cloud)
Expo CLI	Latest	Mobile app development

Software	Version	Purpose
VS Code	Latest	Primary IDE
Google Colab	N/A	ML model training (cloud GPU)

1.3 Machine Learning Technology

AgriGuard utilizes TensorFlow and Keras, leveraging the highly optimized EfficientNetB3 architecture as the core backbone for its dual-stage classification pipeline. Selected specifically for its compound scaling method, EfficientNetB3 achieves superior accuracy with significantly fewer parameters than traditional architectures like ResNet or VGG. The system deploys two uniquely trained models that process 300x300 pixel RGB images: Model 1 serves as the initial Crop Classifier, identifying the presence of 10 supported crops or an "Unknown" class, while Model 2 operates as the primary Disease Diagnoser, evaluating 76 distinct disease conditions and healthy states. To ensure highly available and scalable AI diagnostics, both models are hosted entirely on Hugging Face Spaces and securely served via a dedicated Python Flask server running TensorFlow in CPU inference mode.

1.4 Purpose of Using Deep Learning for Crop Disease Detection

Conventional disease identification requires trained agronomists to physically visit each field, examine leaf symptoms, and provide a diagnosis, a process that may take days and is often unavailable in remote farming communities. Deep learning offers a practical alternative for these reasons: Pattern Recognition in which CNNs can learn subtle visual patterns in leaf lesions, discoloration, and texture changes that distinguish between similar-looking diseases. These patterns are often too subtle for the untrained human eye. And A trained model can classify an image in under two seconds, compared to hours or days for manual expert consultation.

The dual-stage approach (first identifying the crop, then diagnosing the disease within that crop's known conditions) reduces false positives and improves diagnostic accuracy compared to a single flat classifier across all 76 classes.

1.5 Importance for Pakistani Agriculture

AgriGuard directly addresses critical challenges in Pakistan's agricultural sector by providing a scalable, mobile-based solution for rural farmers. It combats the 15-25% annual yield loss and widespread pesticide misuse caused by misdiagnosed plant diseases by putting an AI-driven diagnostic tool in every farmer's pocket, effectively bridging the severe shortage of agricultural extension workers. Furthermore, AgriGuard tackles the country's escalating water scarcity through an automated, moisture-based irrigation system that significantly reduces water wastage, leveraging Pakistan's growing smartphone penetration to deliver accessible, precision farming capabilities nationwide.

1.6 Functionality of AgriGuard

AgriGuard provides the following core functionalities:

1. **User Registration and Authentication:** Farmers create accounts using email-based OTP verification through the Brevo SMTP API, ensuring only legitimate users access the platform. Authentication is maintained using JWT tokens stored securely in the device's native keychain.
2. **Disease Detection:** Users capture or upload leaf images, which are compressed client-side and transmitted to the Flask ML server hosted on Hugging Face Spaces. The dual-stage pipeline identifies the crop species and diagnoses any disease condition, returning results with confidence percentages.
3. **Pesticide Advisory:** Upon disease detection, the system automatically retrieves treatment recommendations from the backend database, including pesticide name, dosage, spray interval, water ratio, and safety instructions.
4. **Scan History:** All diagnostic results are saved to MongoDB Atlas and accessible through a chronological history view, allowing farmers to track crop health over time and delete individual records.
5. **Smart Water Management:** Farmers can remotely monitor soil moisture levels, manually control a water pump, or enable automatic irrigation that activates based on configurable moisture thresholds.
6. **Profile Management:** Users can update their personal information, change passwords, set profile pictures, view diagnostic statistics, and access a comprehensive supported crops catalog.

1.7 Modules

AgriGuard is organized into six distinct functional modules:

Table 3: Core System Modules Overview

Sr. no	Module	Description
1	Authentication & Account Management	Email registration with OTP via Brevo, login with JWT (30-day), password reset, profile editing, password change
2	Crop Disease Detection	Dual-stage EfficientNetB3 inference on Hugging Face Spaces, camera/gallery/ESP32-CAM input, confidence gating with dynamic fallback
3	Pesticide Advisory System	76+ disease-specific treatment records with pesticide name, dosage, spray interval, water ratio, safety instructions, and notes
4	Scan History Management	Create, read, and delete scan records with user-ownership authorization and pagination
5	Smart Water Management	Soil moisture monitoring (0-100%), manual pump control, automatic threshold-based irrigation (<30% ON, >70% OFF)
6	Profile Management	Name editing, password change, profile picture, scan statistics dashboard, supported crops catalog, cache clearing

1.8 Benefits

AgriGuard offers transformative benefits by delivering instant, AI-driven disease diagnosis and targeted pesticide recommendations, drastically reducing response times while promoting sustainable farming practices. Its remote and automated irrigation capabilities save immense physical effort and optimize water usage based on real-time sensor data, with hardware that functions independently of network connectivity. Furthermore, the system empowers farmers with persistent scan histories to track seasonal disease patterns, all delivered through a secure, cross-platform mobile application supported by a robust, 24/7 cloud infrastructure.

1.9 Objectives

The primary objectives of AgriGuard are:

1. To develop a cross-platform mobile application that enables farmers to identify crop diseases by photographing leaf symptoms using their smartphone camera, with results delivered in seconds.
2. To build a dual-stage deep learning pipeline using EfficientNetB3 architecture capable of first identifying the crop species from 11 possible classes and then diagnosing the specific disease condition from 76 possible classes with measurable confidence.
3. Providing actionable pesticide treatment advisories immediately upon diagnosis, including pesticide name, dosage, application frequency, water-to-chemical ratio, and safety precautions.
4. To design and implement an IoT-based smart irrigation system using ESP32 microcontrollers that monitors soil moisture in real-time and controls a water pump both manually via mobile app and automatically based on configurable thresholds.
5. To create a secure authentication system with email-based OTP verification (via Brevo SMTP API), JWT session management, and platform-native encrypted storage for tokens.
6. To maintain a persistent historical record of all diagnostic scans in MongoDB Atlas, enabling farmers to track crop health trends over multiple growing seasons.
7. To deploy the system on cloud infrastructure (Render for backend, Hugging Face Spaces for ML) ensuring continuous availability and scalability.

1.10 Features

1.10.1 Features of the Mobile Application

The AgriGuard mobile application provides a comprehensive, cross-platform interface secured by OTP-verified registration and JWT-based authentication. Users can initiate real-time AI disease analysis using images from their phone or the remote ESP32-CAM, receiving detailed diagnostic scores and comprehensive pesticide advisories. Beyond diagnostics, the app serves as a complete farm management dashboard, offering real-time soil moisture monitoring, manual and threshold-based automated irrigation controls, and persistent scan history tracking. These core functions are enhanced by a highly responsive, animated UI featuring custom navigation, personalized greetings, daily farming tips, and robust profile management.

1.10.2 Features of the IoT Hardware System

The IoT hardware subsystem utilizes an ESP32-CAM for high-resolution, outdoor-optimized leaf photography alongside an ESP32-WROOM for precision environmental control. The system continuously maps raw capacitive sensor data to soil moisture percentages, driving an automated irrigation algorithm that toggles a water pump relay based on optimized moisture thresholds (30%-70%). To ensure maximum field reliability and hardware longevity, the microcontrollers incorporate critical safety failsafes, throttled execution cycles, dynamic IP registration, and disabled brownout detectors to maintain stable, uninterrupted telemetry with the cloud backend.

1.11 Functionality

Core functionalities include real-time leaf image acquisition, dual-stage crop and disease classification, automated pesticide advisory generation, live soil moisture monitoring, and remote irrigation control. The system integrates these capabilities into a unified interface, streamlining farm management operations.

1.12 Purposes & Benefits

The primary purpose of AgriGuard is to bridge the critical gap between limited agricultural extension services and local farmers by delivering a practical, real-world application of deep learning and cloud-integrated IoT hardware. This unified smart farming ecosystem provides immense benefits: it dramatically reduces crop yield losses

through early disease identification, conserves up to 40% of water usage via threshold-based automated irrigation, and saves significant financial resources by preventing the application of incorrect pesticides. Furthermore, AgriGuard maximizes time efficiency through remote hardware control and empowers untrained farmers with expert-level knowledge and historical scan data, allowing them to make informed, data-driven decisions regarding long-term crop management.

1.13 Introduction to the Machine Learning Pipeline

The AgriGuard machine learning pipeline utilizes the EfficientNetB3 architecture, selected for its optimal balance between model size and high diagnostic accuracy on CPU-based servers. Leveraging advanced features like 300x300 input resolution, depthwise separable convolutions, and Squeeze-and-Excitation blocks, the model was fine-tuned specifically for agricultural diagnostics. To prevent inter-crop confusion, the system employs a unique dual-stage inference pipeline: Stage 1 first identifies the plant among 11 supported categories, while Stage 2 pinpoints the exact condition by evaluating probabilities exclusively filtered for that specific crop. This robust architecture incorporates dynamic fallback mechanisms to resolve ambiguous leaf features and enforces strict confidence thresholds, guaranteeing highly reliable and accurate disease predictions for farmers.

1.14 Introduction to the Operating Environment

The AgriGuard operating environment integrates a robust, distributed technology stack designed for maximum scalability and cross-platform accessibility. The mobile frontend is built using React Native and the Expo SDK, ensuring native performance and seamless hardware integration across Android and iOS devices. This client communicates securely with a Node.js and Express backend API deployed on Render, which manages JWT authentication, handles high-volume OTP email delivery via the highly reliable Brevo REST API, and interfaces with MongoDB Atlas for flexible, cloud-hosted data persistence. Machine learning operations are intentionally offloaded to a dedicated Python and Flask inference server hosted on Hugging Face Spaces, isolating heavy computational tasks. Finally, the field-deployed IoT hardware relies on custom C++ firmware compiled via the Arduino framework, providing the ESP32 microcontrollers

with stable WiFi connectivity, sensor management, and secure HTTPS telemetry to complete the end-to-end smart farming ecosystem.

1.15 Tools Used to Develop AgriGuard

The development of AgriGuard relied on a suite of industry-standard tools to ensure robust full-stack integration. Visual Studio Code served as the primary IDE for mobile app, backend and flask API, while the Arduino IDE was utilized for compiling C++ firmware for the ESP32 microcontrollers. The EfficientNetB3 machine learning models were trained via Google Colab Pro. Database management and API endpoint verification were streamlined using MongoDB Compass and Postman, with all project version control and APK distribution centrally managed through Git and GitHub.

1.16 IoT Hardware Components

The physical deployment utilizes an ESP32-CAM module equipped with an OV2640 sensor for remote crop monitoring, and an ESP32-WROOM microcontroller paired with an analog capacitive soil moisture sensor. A 5V submersible water pump, controlled via a single-channel isolation relay, executes the automated irrigation commands.

Chapter 2

Software Requirements Specifications

2 Software Requirements Specifications

2.1 Introduction of the Requirements

Requirements engineering is a foundational phase in software development that establishes what a system must do (functional requirements) and how well it must perform (non-functional requirements). For AgriGuard, requirements were gathered through a combination of domain research into agricultural challenges in Pakistan, analysis of existing plant disease detection applications, study of agricultural science literature on crop diseases, and iterative hardware prototyping with ESP32 modules.

The requirements presented in this chapter represent the final, implemented specification of the AgriGuard system as verified against the production codebase. Each requirement has been confirmed to exist in the code through line-level inspection of the relevant source files.

2.2 Software Requirement Specification (SRS)

A Software Requirement Specification (SRS) document defines the complete set of functionalities and constraints that a system must satisfy. It serves as a contract between the development team and stakeholders, establishing a shared understanding of what will be built.

The SRS for AgriGuard is particularly complex because it spans four technology tiers: mobile application, backend server, ML inference engine, and IoT hardware, each with its own constraints and capabilities.

2.3 Goals of Software Requirement Specification

The goals of the AgriGuard SRS are to:

1. Establish a clear, unambiguous description of all system functionalities that can be directly verified against the implementation
2. Define the interfaces between the mobile app, Express backend, Flask ML server, MongoDB database, Brevo email API, and ESP32 IoT hardware
3. Specify security requirements for user authentication, data protection, and secure token management

4. Set measurable performance expectations including ML confidence thresholds, polling intervals, and upload size limits
5. Serve as the primary reference for verification and validation testing throughout the development lifecycle

2.4 Gathering Requirements

System requirements were gathered through domain research into agricultural inefficiencies, technical feasibility studies regarding ESP32 hardware limitations, and iterative feedback loops concerning user interface accessibility for rural farming communities.

2.4.1 Requirements Gathering Process

1. **Identification:** Initial feature list derived from project objectives and stakeholder expectations. Each proposed feature was documented with a brief description and priority level.
2. **Analysis:** Feasibility assessment for each requirement considering available hardware (ESP32 module capabilities), ML model limitations (EfficientNetB3 capacity), third-party service constraints (Brevo API limits, Render free tier), and development timeline.
3. **Specification:** Detailed documentation of each accepted requirement with formal acceptance criteria, including specific threshold values (e.g., confidence of 0.45 (45%) for crop classification) and measurable parameters.
4. **Validation:** Cross-referencing requirements against the implemented system to ensure completeness. Each requirement was traced to specific source code files and line numbers.
5. **Prioritization:** Classification into P0 (must-have system cannot function without this) and P1 (nice-to-have enhances user experience but system works without it).

2.5 Capturing Better Requirements

To guarantee requirement quality and eliminate ambiguity, AgriGuard adhered to five strict engineering principles. First, all requirements were kept atomic, describing single, independently testable capabilities to prevent compound confusion. Second, strict traceability matrices linked every requirement directly to specific code modules for comprehensive verification. Third, requirements utilize objective, measurable criteria

with specific numeric thresholds rather than subjective descriptions. Fourth, boundary conditions and edge cases were explicitly defined to eliminate developer guesswork during error handling. Finally, all requirements were written using user-centric language, ensuring the final system remained intensely focused on human experience.

2.6 Different Types of Software Requirements

Requirements are categorized into functional necessities, such as image uploading and hardware toggling, and non-functional constraints, including system security, performance latency, and cross-platform UI responsiveness. These specifications form the baseline for system validation.

2.6.1 Functional Requirements

2.6.1.1 User Registration & Authentication

Table 4: Functional Requirements - User Authentication

ID	Requirement	Priority
FR-AUTH-01	The system shall allow users to register with name, email address, and password	P0
FR-AUTH-02	Registration shall require email verification via a 6-digit OTP code delivered through the Brevo SMTP API	P0

2.6.1.2 Crop Disease Detection

Table 5: Functional Requirements - Dashboard & Profile

ID	Requirement	Priority
FR-DET-01	Users shall be able to capture leaf images using the device camera with optional editing/cropping	P0
FR-DET-02	Users shall be able to select leaf images from the device photo gallery with optional editing	P0

2.6.1.3 Pesticide Advisory

Table 6: Functional Requirements - Disease Detection

ID	Requirement	Priority
FR-PEST-01	The system shall automatically fetch pesticide advisory from the backend after disease detection using the disease_label as lookup key	P0
FR-PEST-02	Each advisory shall include six fields: pesticide name, dosage, spray interval, water-to-chemical ratio, safety instructions, and additional notes	P0

2.6.1.4 Scan History Management

Table 7: Functional Requirements - Irrigation Control

ID	Requirement	Priority
FR-HIST-01	Each scan result shall be saved to MongoDB with user reference, crop, disease, confidence, is_healthy flag, and optional image URI	P0
FR-HIST-02	Users shall view their scan history sorted by newest first (createdAt descending)	P0

2.6.1.5 Smart Water Management

Table 8: Functional Requirements - Crop Catalog & History

ID	Requirement	Priority
FR-WATER-01	The app shall display real-time soil moisture percentage, updated by polling the ESP32-WROOM status every 4 seconds	P0
FR-WATER-02	Users shall be able to manually turn the water pump ON and OFF through the mobile app	P0

2.6.1.6 Profile Management

Table 9: Functional Requirements - Offline Sync

ID	Requirement	Priority
----	-------------	----------

ID	Requirement	Priority
FR-PROF-01	Users shall view their scan statistics showing three counts: total scans, diseased detections, and healthy detections	P1
FR-PROF-02	Users shall access a comprehensive catalog of 10 supported crops and their associated diseases through a bottom sheet modal	P1

2.6.2 Non-Functional Requirements

AgriGuard strictly adheres to comprehensive non-functional requirements to ensure a premium, secure, and robust user experience. For usability and user-friendliness, the cross-platform interface features intuitive 3-tap navigation, animated transitions, in-app image cropping, and clear, non-technical error handling. Reliability and performance are guaranteed through critical hardware failsafes, hysteresis-gapped auto-watering algorithms, client-side image compression, and optimized polling intervals that balance system responsiveness with sensor longevity. Security is uncompromising, utilizing bcrypt hashing, platform-specific JWT encrypted storage, strict middleware validation, and ownership authorization. Finally, the system's highly scalable architecture decouples the Node.js API, MongoDB database, and dual-model Python inference server, complemented by a dynamic IoT device registry that seamlessly supports future hardware expansion without requiring code modifications.

2.8 Deployment Requirements

Table 10: Non-Functional Requirements Implementation

Requirement	Implementation
Backend shall be deployed on a cloud platform with HTTPS	Render (https://agriguard-g3hq.onrender.com)
ML server shall be deployed with sufficient resources for TensorFlow inference	Hugging Face Spaces (https://mearslanahmed-agriguard-flask.hf.space)
Database shall be hosted on a managed cloud service with automated backups	MongoDB Atlas (cloud cluster with replication)

Chapter 3

Software Design Description

3 Software Design Description

3.1 Methodology

AgriGuard was developed following an iterative development methodology, where each module (authentication, disease detection, irrigation, etc.) was designed, implemented, tested, and refined in successive cycles. This approach allowed early detection of integration issues particularly between the mobile app, backend server, Flask ML server, and ESP32 hardware and enabled requirements to be adjusted based on practical constraints discovered during development.

The design phase produced several UML and system diagrams to document the architecture, data flow, and interaction patterns before implementation. All diagrams presented in this chapter have been verified against the final production codebase.

3.2 Unified Modeling Language (UML)

In this project, the Unified Modeling Language (UML) is utilized to meticulously specify, visualize, and document the system's structural and behavioral architecture. Created using StarUML and draw.io, the documentation relies on a comprehensive suite of visual models: Use Case diagrams define the functional scope and actor interactions, Sequence diagrams illustrate the precise temporal flow of cross-component messaging, Activity diagrams map out complex step-by-step workflows like the AI disease detection pipeline, and Entity-Relationship diagrams clearly outline the fundamental database schemas and data collection relationships.

3.3 Designing Technique

The AgriGuard system design utilizes a highly decoupled, four-tier microservice architecture that enables independent development, deployment, and scalability. The Client Tier features a React Native mobile application managing the UI and local state, while the API Gateway Tier employs a Node.js and Express server for RESTful authentication, data persistence, and hardware proxying. Heavy computational tasks are isolated in the ML Inference Tier using a dedicated Python and Flask server for TensorFlow predictions, and physical farm operations are managed by the IoT Hardware Edge Tier via ESP32 microcontrollers for sensor telemetry and actuator control. To

maximize operational performance, the mobile application communicates directly with the Flask server for rapid AI inference, efficiently bypassing the primary API gateway.

3.4.1 Why We Used Agile Development

The Agile iterative model was strategically selected to accommodate the project's complex hardware-software integration and evolving technical requirements. It allowed for crucial multiple iterations to establish reliable reverse proxy communication and dynamic IP registration between the ESP32 hardware and the cloud backend. Furthermore, Agile facilitated repeated cycles of machine learning model training and strict confidence threshold tuning, enabling the team to pivot from an initial flat classifier design to a far superior dual-stage diagnostic pipeline. By continuously delivering testable subsets of features across both mobile and hardware platforms, the Agile approach ensured immediate, real-world feedback throughout the entire development lifecycle.

3.4.2 Development Phases

The project progressed through the following iterations:

Table 11: Project Timeline and Deliverables

Phase	Duration	Deliverables
Phase 1	Weeks 1-3	Project setup, ML dataset collection, initial model training
Phase 2	Weeks 4-6	Flask inference server, single-stage detection prototype
Phase 3	Weeks 7-9	Express backend with auth, database schema design, mobile app skeleton
Phase 4	Weeks 10-12	Dual-stage ML pipeline, dynamic fallback, pesticide advisory system
Phase 5	Weeks 13-15	ESP32 firmware, IoT integration, water management module
Phase 6	Weeks 16-17	UI polish, testing, bug fixes, documentation

3.5 System Features

3.5.1 Disease Detection System

The disease detection system accepts leaf images through three input channels (phone camera, gallery, ESP32-CAM) and processes them through a dual-stage EfficientNetB3 pipeline. Stage 1 identifies the crop species from 11 possible classes, and Stage 2 diagnoses the specific disease condition from 76 classes. The complete disease detection activity flow is documented in the activity diagram.

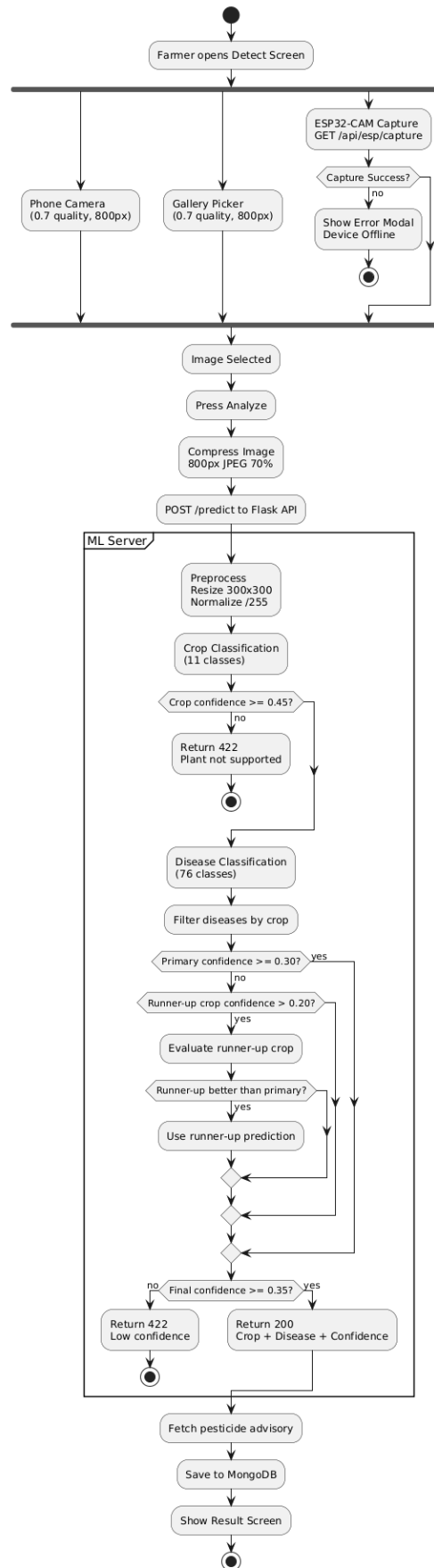


Fig. 1: Activity Diagram - Disease Detection Flow

3.5.2 Smart Water Management System

The smart water management system consists of an ESP32-WROOM microcontroller connected to a capacitive soil moisture sensor and a relay-controlled water pump. The system supports two operational modes: manual (direct pump control from the mobile app) and automatic (threshold-based irrigation with moisture < 30% triggering pump activation and > 70% triggering deactivation).

3.5.3 Pesticide Advisory System

Upon disease diagnosis, the mobile app queries the backend for a matching pesticide advisory record. The backend performs a case-insensitive regex lookup in the Pesticide collection using the `disease_label` from the ML prediction.

3.6 Use Case Diagram

The AgriGuard use case diagram meticulously outlines the interactions between five primary actors across 20 distinct system use cases, categorized into Authentication, Disease Detection, Scan History, Smart Water Management, and Profile Management. The primary human actor is the Farmer, who interfaces directly with the mobile application. The system also seamlessly integrates four autonomous actors: the ESP32-CAM hardware node for on-demand image capture, the ESP32-WROOM controller for managing soil moisture and pump operations, the Flask ML Server acting as the core AI inference engine for disease predictions, and the external Brevo SMTP API managing the secure delivery of OTP verification codes.

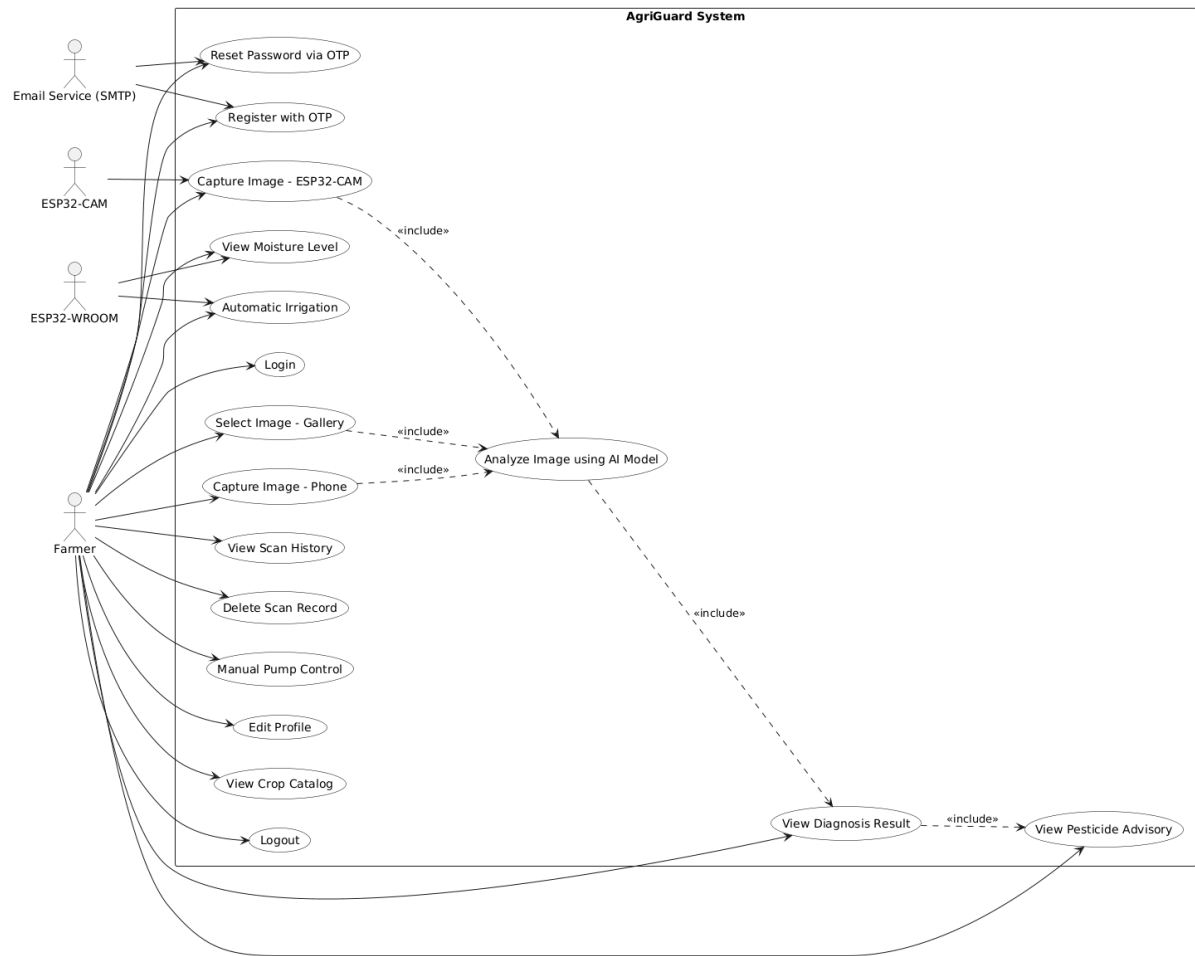


Fig. 2: Use Case Diagram

3.6.1 Use Case: Farmer Detects Disease

Table 12: Sequence Diagram - Authentication Flow

Element	Description
Actor	Farmer
Precondition	Farmer is logged in; device has internet connectivity
Main Flow	- Farmer selects image source (camera/gallery/ESP32-CAM) - Image is captured/selected - Farmer presses "Analyze Crop" - Image is compressed and sent to Flask ML server - Dual-stage prediction is performed

Element	Description
	6. Result and pesticide advisory are displayed
Alternate Flow	If crop confidence < 0.45 or disease confidence < 0.35, an error modal is shown
Postcondition	Scan result is saved to database; farmer views diagnosis with treatment advisory

3.6.2 Use Case: Farmer Controls Irrigation

Table 13: Sequence Diagram - Disease Scan Flow

Element	Description
Actor	Farmer, ESP32-WROOM
Precondition	Farmer is logged in; ESP32-WROOM is online and registered
Main Flow	1. Farmer opens Water Management screen 2. App polls device status every 4 seconds 3. Farmer toggles pump ON/OFF or enables auto mode 4. Confirmation modal is shown 5. Command is sent via backend proxy to ESP32-WROOM 6. Device executes command and returns status
Alternate Flow	If ESP32-WROOM is offline, an offline indicator is shown and controls are disabled
Postcondition	Pump state is changed; moisture reading is updated

3.6.3 Use Case: Farmer Manages Account

Table 14: Sequence Diagram - Hardware Sync Flow

Element	Description
Actor	Farmer, Brevo SMTP API
Precondition	None (for registration/login); logged in (for profile management)

Element	Description
Main Flow	1. Farmer fills registration form 2. System sends 6-digit OTP to email 3. Farmer enters OTP 4. Account is created and JWT is issued 5. Farmer can later update name, change password, or reset password via OTP
Postcondition	User account exists in database; JWT token stored in secure storage

3.7 Entity Relationship Diagram

The ERD maps the relational flow between core entities: Users, Devices, Scans, and Advisories. It outlines a flexible NoSQL structure where users maintain one-to-many relationships with historical scan records and registered hardware nodes.

3.7.2 Database Schema Description

1. **User:** Stores farmer account information including bcrypt-hashed password, with a `matchPassword()` instance method for credential verification. Has a pre-save hook that automatically hashes passwords before persistence.
2. **Scan:** Records each disease detection result, linked to the owning User via an `ObjectId` foreign key. Stores crop name, disease name, confidence percentage, health status, and an optional local image URI.
3. **Pesticide:** Contains 76+ treatment advisory records, each with a unique `class_name` matching the Flask ML server's `disease_label` output. Includes an embedded pesticide subdocument with name, dosage, spray interval, water ratio, safety instructions, and notes.
4. **OTP:** Temporary storage for email verification codes. Uses a MongoDB TTL index on `createdAt` with a 300-second (5-minute) expiration, causing automatic document deletion.
5. **EspDevice:** Registry of IoT hardware nodes. Each ESP32 registers its `deviceId` and current ip address on boot, enabling the backend to locate and proxy requests to the correct device.

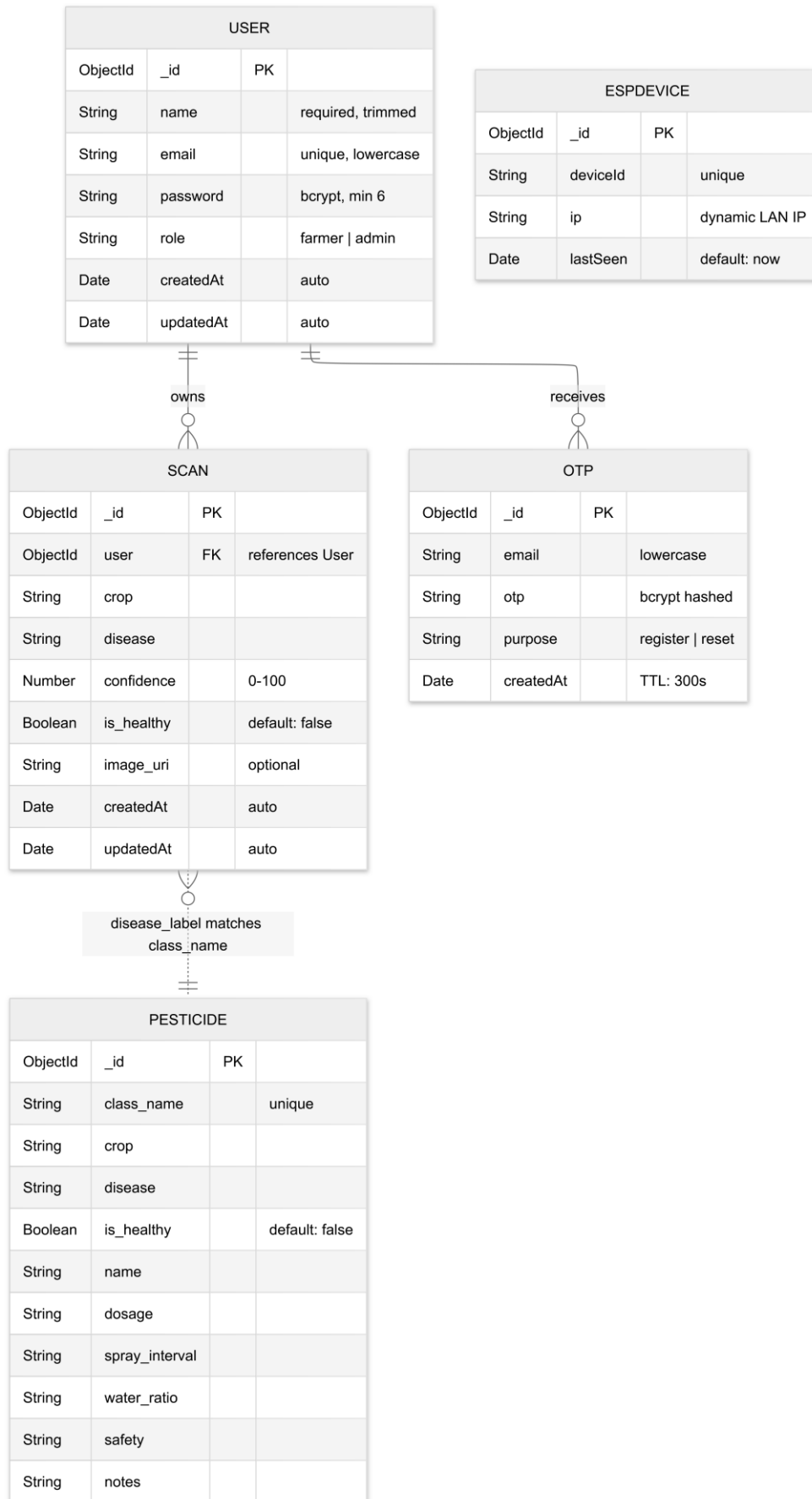


Fig. 3: Entity Relationship Diagram

3.8 System Sequence Diagrams

System sequence diagrams illustrate the chronologically ordered interactions between the mobile app, backend servers, AI models, and IoT devices. Critical flows mapped include the secure user authentication process, the multipart image upload and dual-stage AI disease prediction cycle, and the bi-directional telemetry loop for automated pump control.

3.8.3 Authentication Sequence

The authentication sequence diagram covers three flows: Registration, Login, and Password Reset.

Registration Flow: The farmer submits name, email, and password. The backend verifies the email is not already registered, generates a 6-digit OTP, hashes it with bcrypt, saves it to the OTP collection with a 5-minute TTL, and sends it via the Brevo SMTP API. The farmer enters the received code, which is validated against the stored hash. Upon success, a User document is created with a bcrypt-hashed password, the OTP is deleted, a 30-day JWT is generated, and the token is stored in SecureStore.

Login Flow: The backend looks up the user by email, retrieves the password hash (excluded from default queries via select: false), and performs a bcrypt comparison. On match, a JWT is issued.

Password Reset Flow: Similar to registration OTP flow, but the purpose is set to "reset." After OTP validation, the user's password is updated (triggering the pre-save bcrypt hook) and the OTP is deleted.

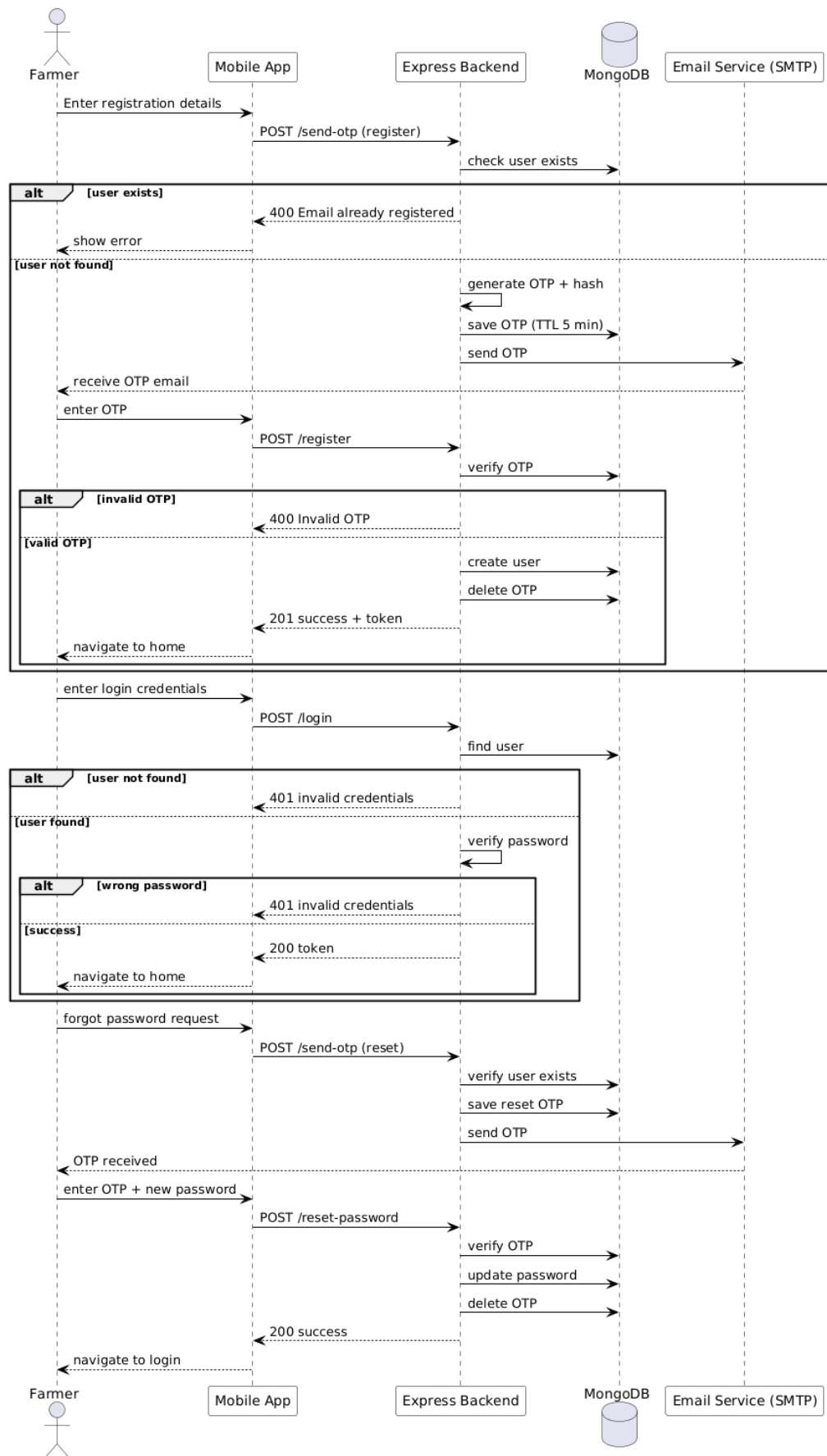


Fig. 4: Sequence Diagram - Authentication Flows

3.8.4 Disease Detection Sequence

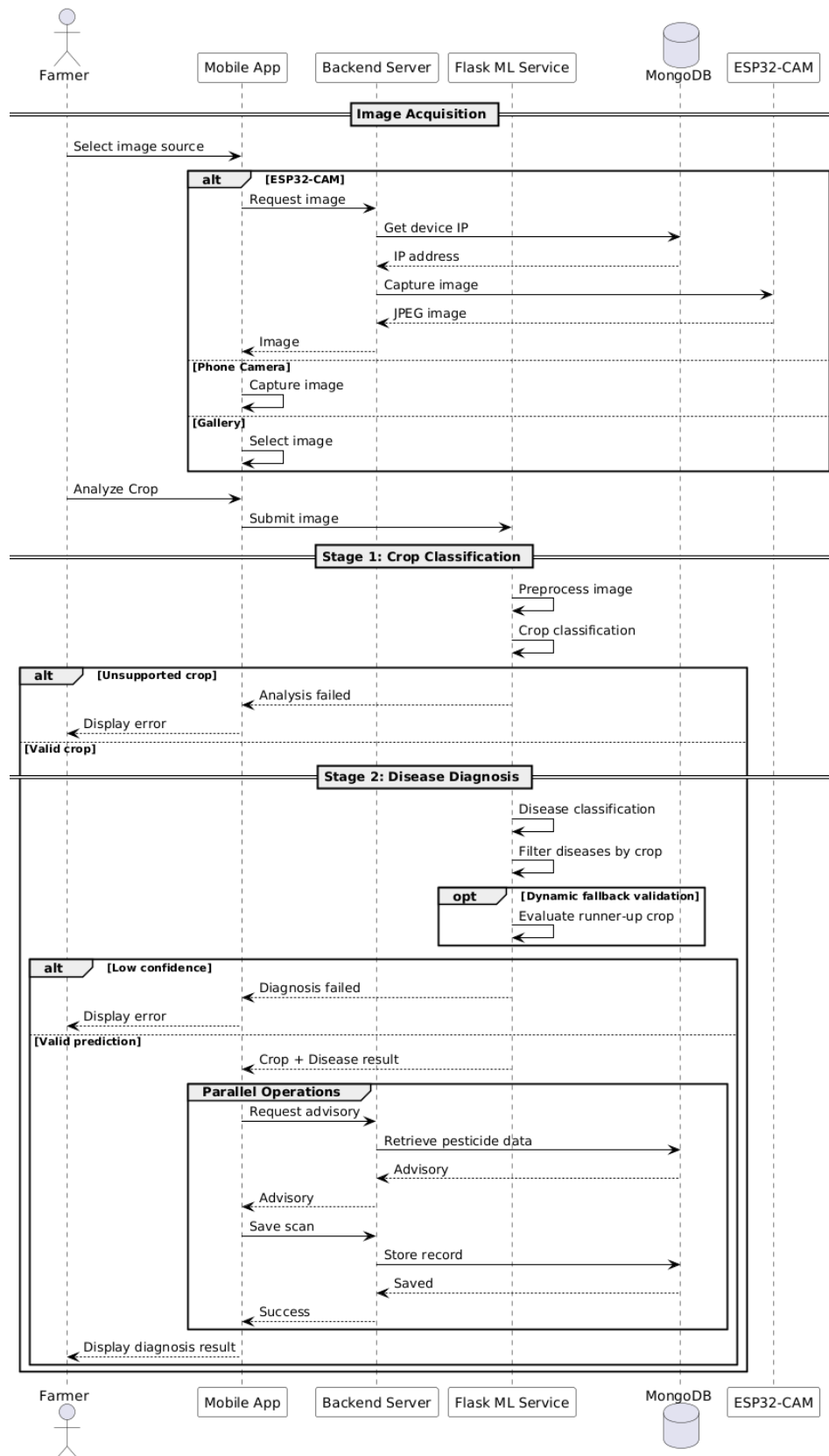


Fig. 5: Sequence Diagram - Disease Detection Data Flow

3.8.5 Water Management Sequence

1. **Boot Sequence:** ESP32-WROOM initializes GPIO pins (relay HIGH for pump OFF, moisture sensor input), connects to WiFi, and sends a POST `/api/esp/register` request with its dynamic IP. The backend performs an upsert on the `EspDevice` collection.
2. **Status Polling:** The mobile app polls GET `/api/esp/wroom/status` every 4 seconds. The backend resolves the device IP from MongoDB and proxies the request. The ESP32 reads the moisture sensor, calculates the percentage, and returns pump state, moisture level, and auto mode status.
3. **Manual Pump Control:** The farmer presses the pump toggle, confirms via modal, and the app sends GET `/api/esp/wroom/pump/on` (or `/off`). The backend proxies to the ESP32, which sets the relay pin accordingly.
4. **Auto Watering:** When enabled, the ESP32 checks moisture every 5 minutes. If below 30%, the pump activates; if above 70%, it deactivates. Disabling auto mode triggers a safety pump-OFF command.

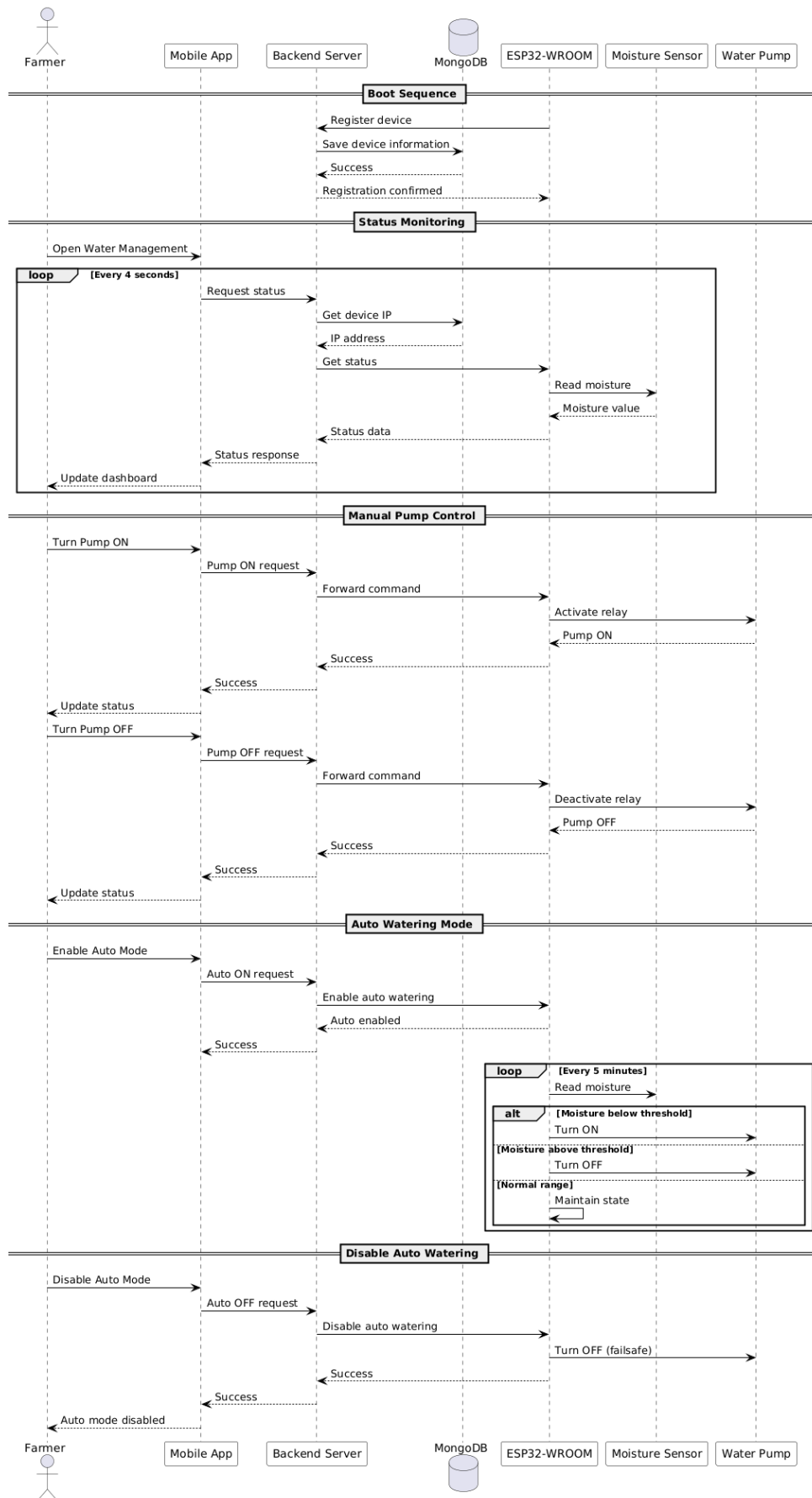


Fig. 6: Sequence Diagram - Smart Water Management Flow

3.9 System Architecture Diagram

AgriGuard is built upon a highly decoupled, 4-tier microservice architecture to ensure independent scalability and service isolation. The Client Tier, developed in React Native and distributed as an APK via GitHub Releases, utilizes 10 navigated screens and SecureStore JWT management to interface seamlessly with the backend. The API Gateway Tier, hosted on Render using Node.js and Express, acts as the central hub, providing JWT authentication, Brevo SMTP integration, dynamic hardware proxying, and robust routing across four dedicated controller modules. Heavy processing is isolated in the ML Inference Tier, a Python and Flask environment hosted on Hugging Face Spaces, which serves dual 125MB Keras models via Gunicorn to handle complex image predictions. Finally, this ecosystem is supported by a secure MongoDB Atlas Data Tier managing five essential collections, alongside a resilient IoT Hardware Edge Tier consisting of ESP32 microcontrollers configured with dedicated HTTP endpoints for remote image capture and automated pump control.

3.9.1 Communication Patterns

Table 15: Software Design Patterns

Pattern	Description
Dynamic IP Registration	ESP32-WROOM registers its LAN IP with the backend on boot via POST /api/esp/register
Reverse Proxy Gateway	Backend resolves device IP from MongoDB, proxies HTTP requests to ESP32 hardware
Direct ML Inference	Mobile app posts images directly to Flask (bypasses Express) for lower latency
Non-blocking Scan Save	After ML prediction, scan is saved to backend in a fire-and-forget try/catch
JWT Token Injection	Axios interceptor on api.js fetches token from SecureStore for every request

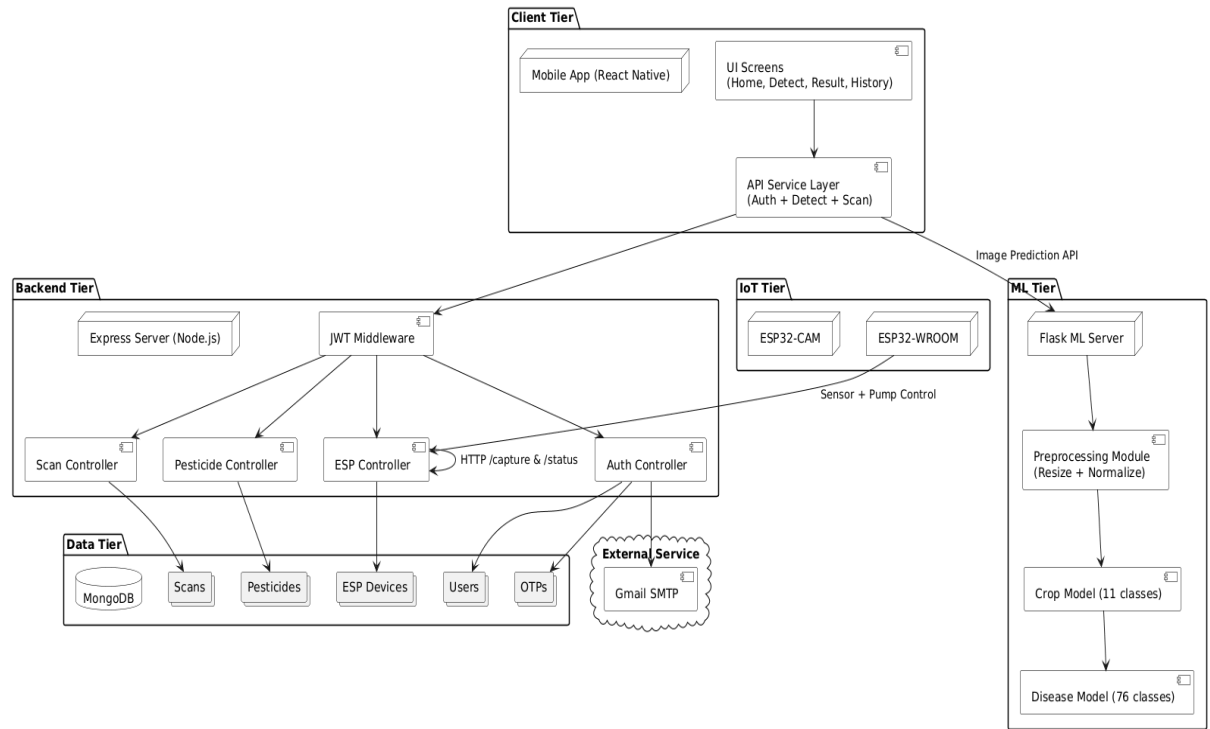


Fig. 7: System Architecture Diagram

3.10 ML Pipeline Diagram

The machine learning pipeline meticulously governs the complete inference flow from initial image upload to the final prediction output. It begins with strict input validation to enforce a 5MB file limit, followed by extensive image preprocessing, including RGB conversion, 300x300 resizing, and pixel normalization, using the PIL library. Stage 1 classification leverages an EfficientNetB3 model to identify the specific crop among 11 classes, instantly rejecting images if the top confidence falls below 45%. Stage 2 then evaluates 76 disease classes, explicitly filtering the probabilities against the crop identified in Stage 1. To maximize accuracy in ambiguous leaf samples, an intelligent dynamic fallback mechanism can strategically pivot to a runner-up crop if the primary disease confidence drops below 30%. Ultimately, a strict final gate requires a minimum 35% disease confidence before constructing and returning the successful diagnostic payload to the user.

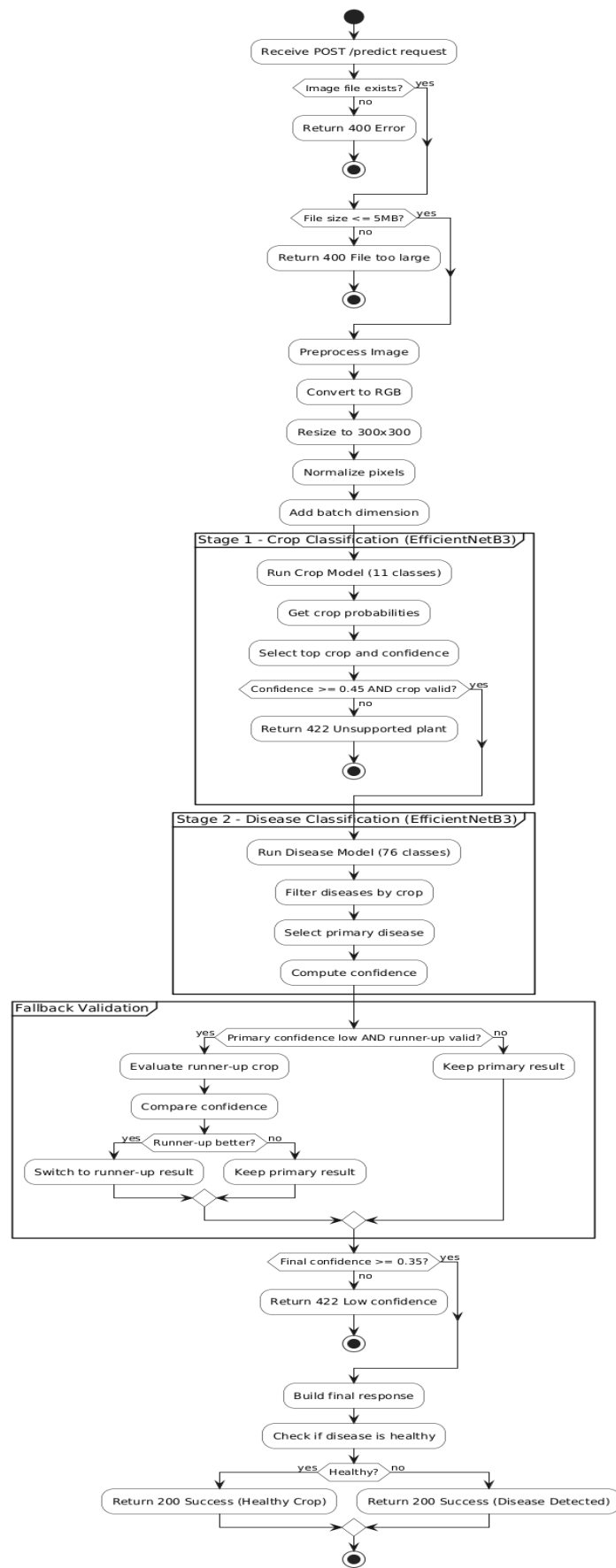


Fig. 8: ML Pipeline Diagram Dual-Stage Inference Flow

3.11 Hardware Communication Diagram

3.11.1 IoT Data Flow

1. **ESP32-CAM Communication:** When the mobile application requests an image, it sends a JWT-authenticated command to the Express backend. The backend dynamically resolves the camera's local IP address from the device registry and securely proxies the request directly to the hardware. The ESP32-CAM then activates its OV2640 sensor, captures a high-resolution JPEG frame, and returns the binary data through the backend proxy directly to the user's mobile interface.
2. **ESP32-WROOM Communication & Automation:** For irrigation management, the mobile application transmits authenticated control commands to the backend, which resolves the ESP32-WROOM's dynamic IP address and proxies the instructions to the hardware. Upon receiving a command, the ESP32-WROOM executes the precise action, such as toggling the 5V relay switch, and immediately returns its execution status. Additionally, the microcontroller runs a completely autonomous auto-watering loop internally every five minutes; it reads GPIO moisture data to toggle the pump at predefined 30% and 70% thresholds entirely independently, requiring no external network communication to function.

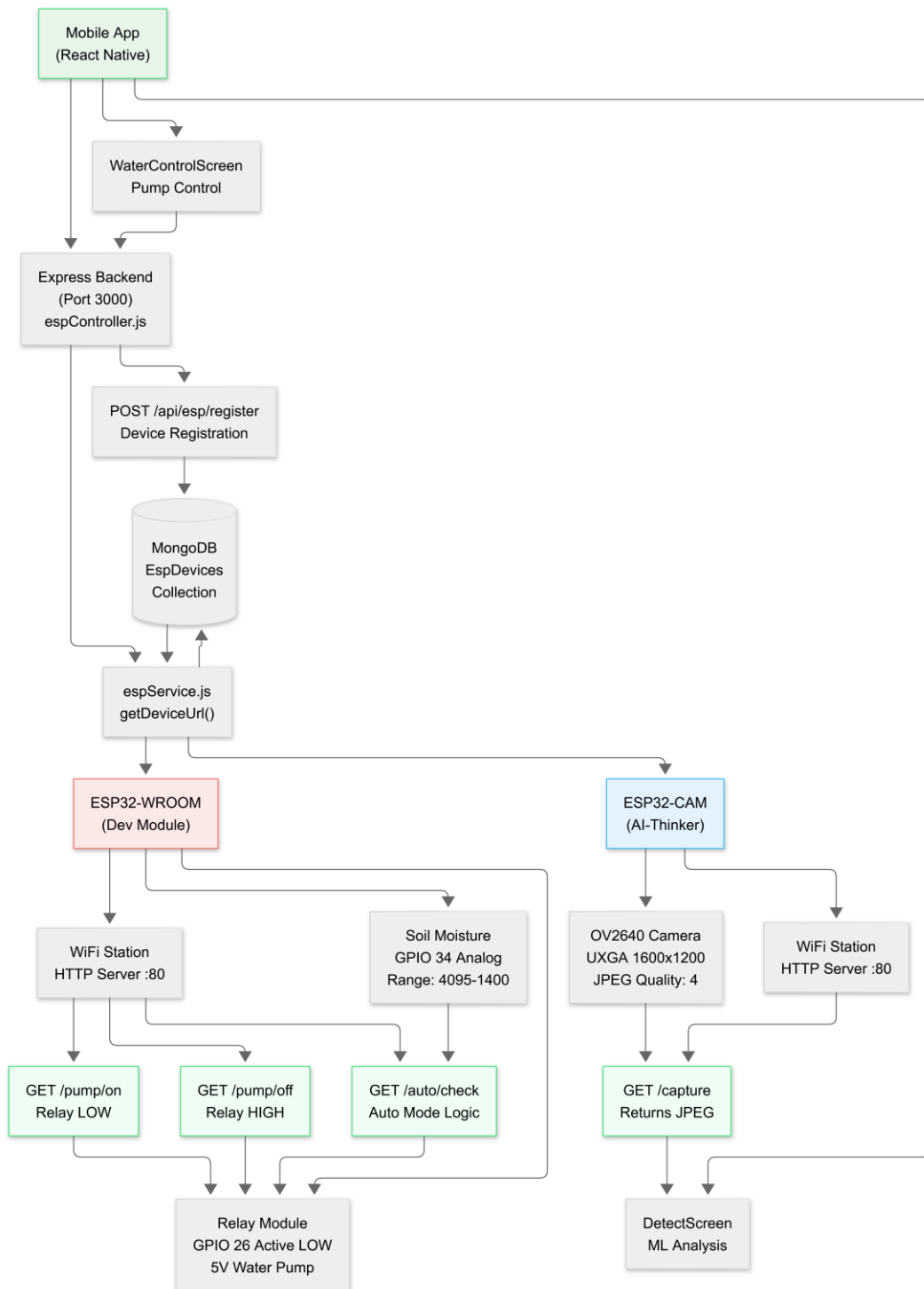


Fig. 9: Hardware Communication Diagram

Chapter 4

Software Development & Implementation

4 Software Development & Implementation

This chapter presents the complete implementation details of AgriGuard which was done using VS Code in React Native, Expo. It covers the mobile application's user-facing screens with screenshots, the complete backend API source code, the Flask ML inference pipeline, ESP32 hardware firmware, database models, the Brevo email integration, and the mobile service layer.

4.1 User Guidelines for AgriGuard

The AgriGuard application is designed for intuitive operation. Users begin by registering an account and securely logging in.

4.1.1 Splash Screen

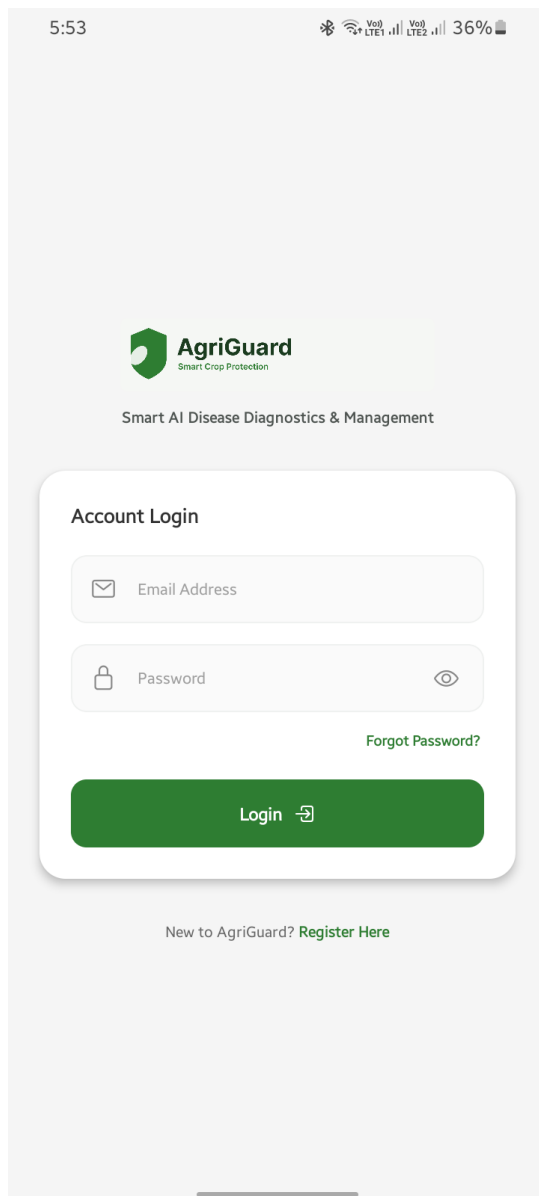
5:50 



AgriGuard

Fig. 10: Splash Screen

4.1.2 Login & Registration Screen



5:53 VoLTE1 VoLTE2 36%

AgriGuard
Smart Crop Protection

Smart AI Disease Diagnostics & Management

Account Login

Email Address

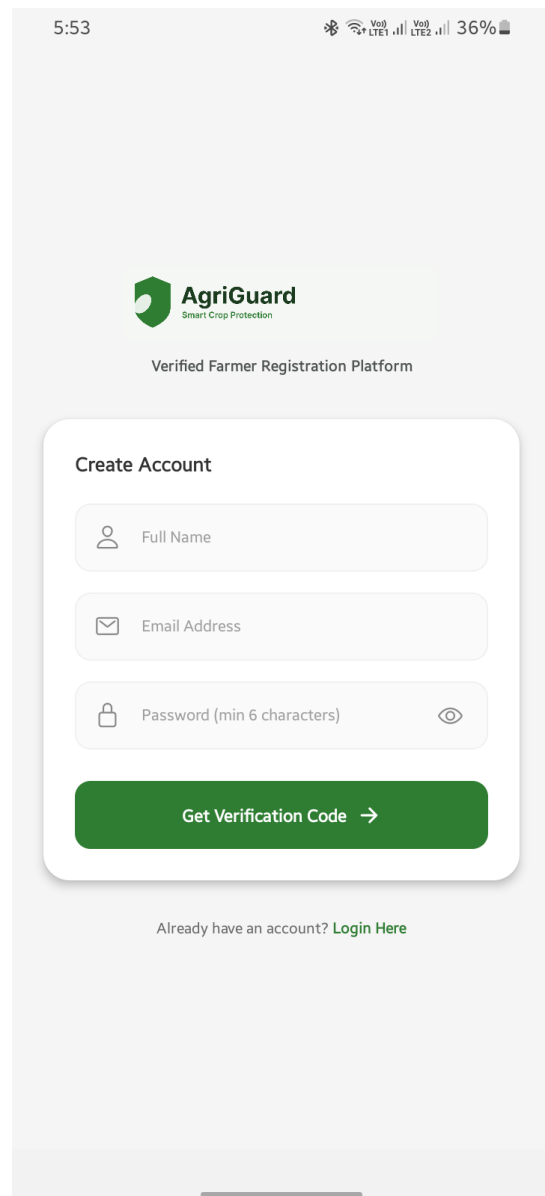
Password 

[Forgot Password?](#)

Login 

New to AgriGuard? [Register Here](#)

Fig. 11: Login Screen



5:53 VoLTE1 VoLTE2 36%

AgriGuard
Smart Crop Protection

Verified Farmer Registration Platform

Create Account

 Full Name

 Email Address

 Password (min 6 characters) 

Get Verification Code 

Already have an account? [Login Here](#)

Fig. 12: Registration Screen

4.1.3 Home Screen (Dashboard)

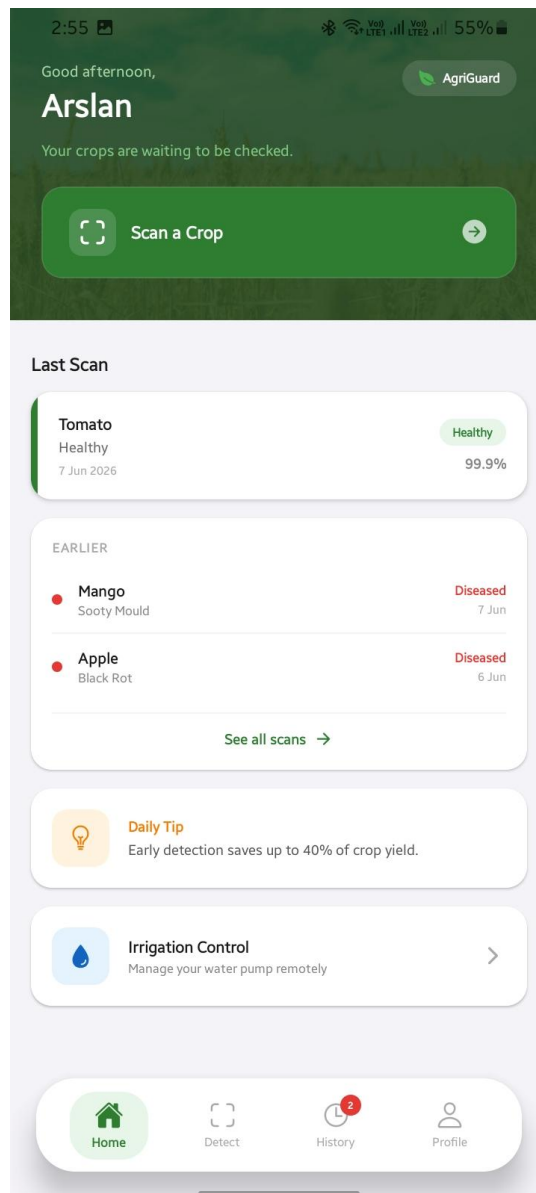


Fig. 13: Home Screen

4.1.4 Detect Screen

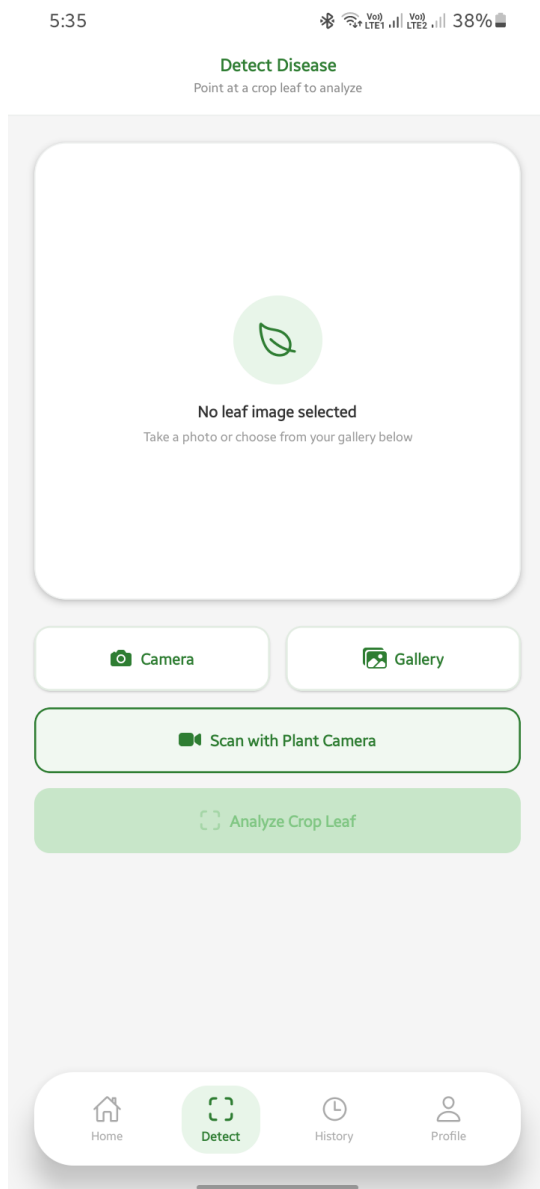


Fig. 14: Detect Screen (Empty State)

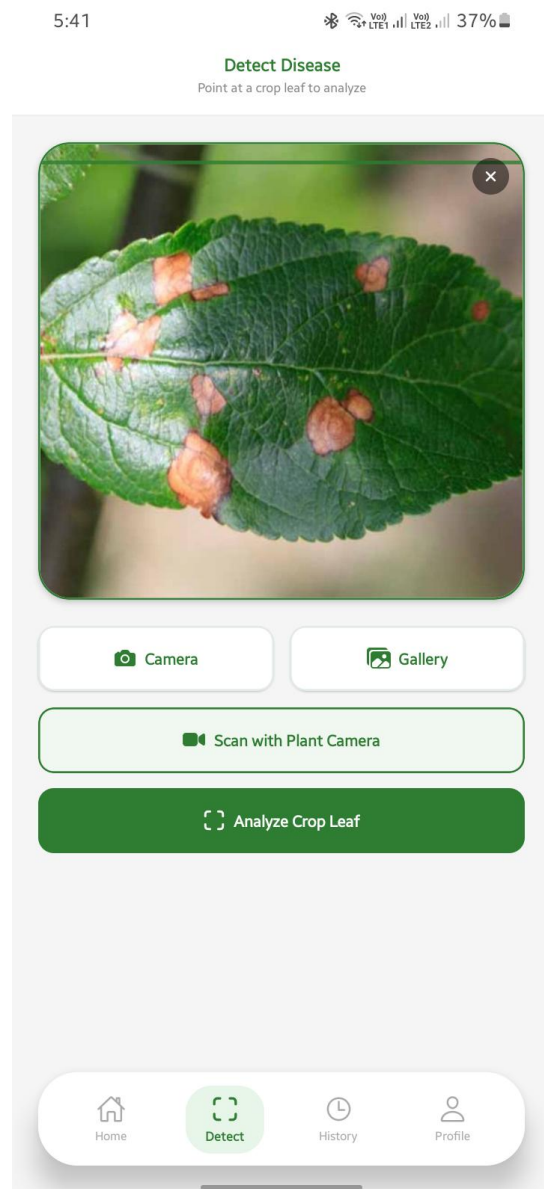


Fig. 15: Detect Screen (Image Selected)

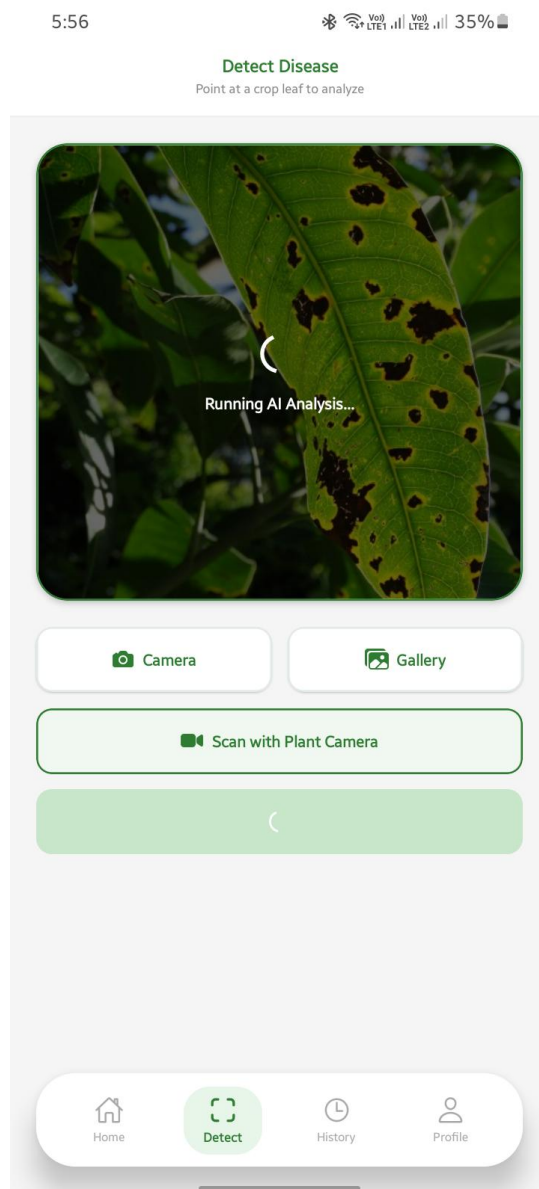


Fig. 16: Detect Screen (Analyzing)

4.1.5 Result Screen

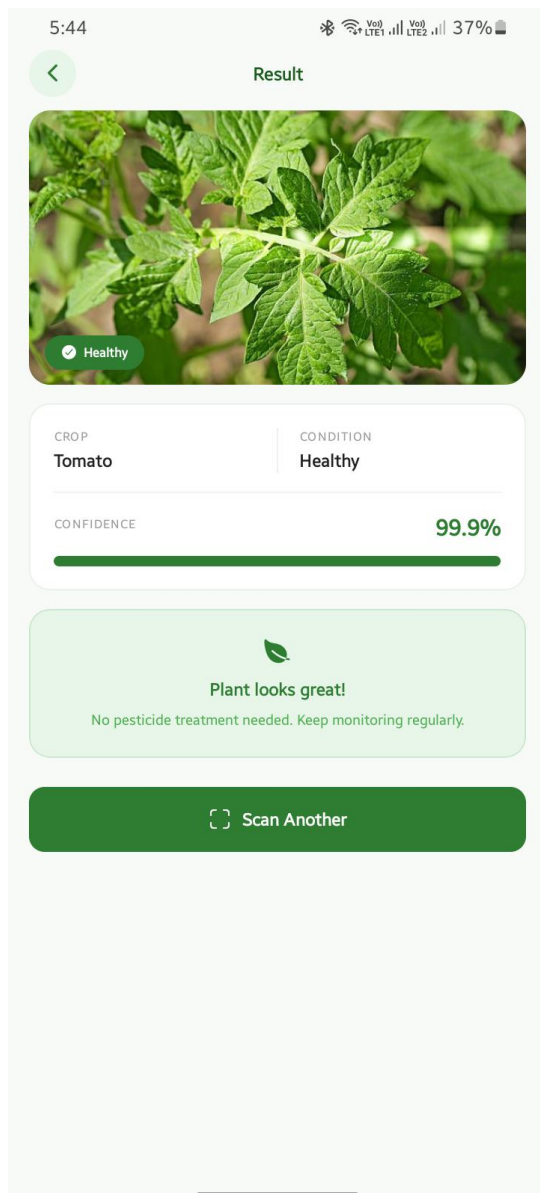


Fig. 17: Result Screen (Healthy Plant)

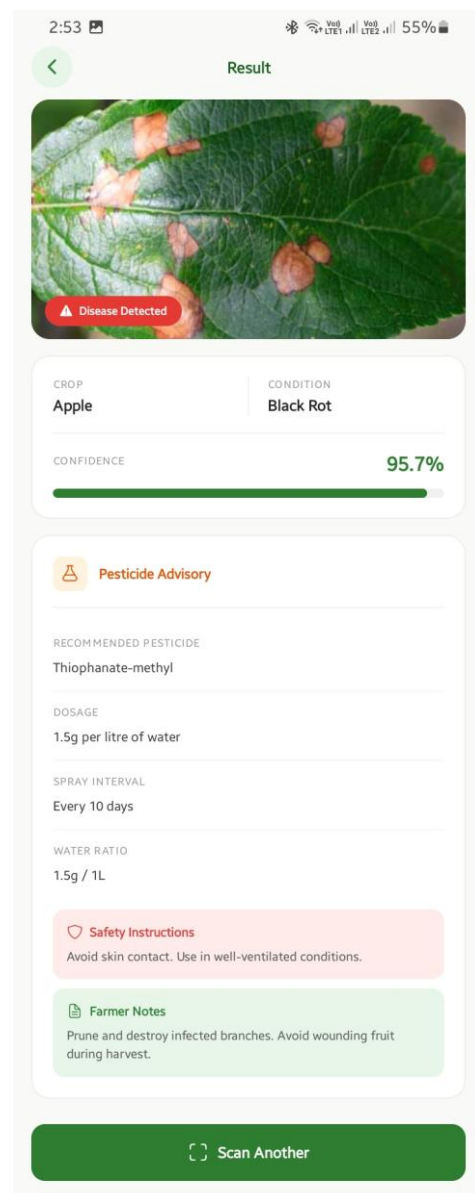


Figure 18: Result Screen (Diseased Plant)

4.1.6 History Screen

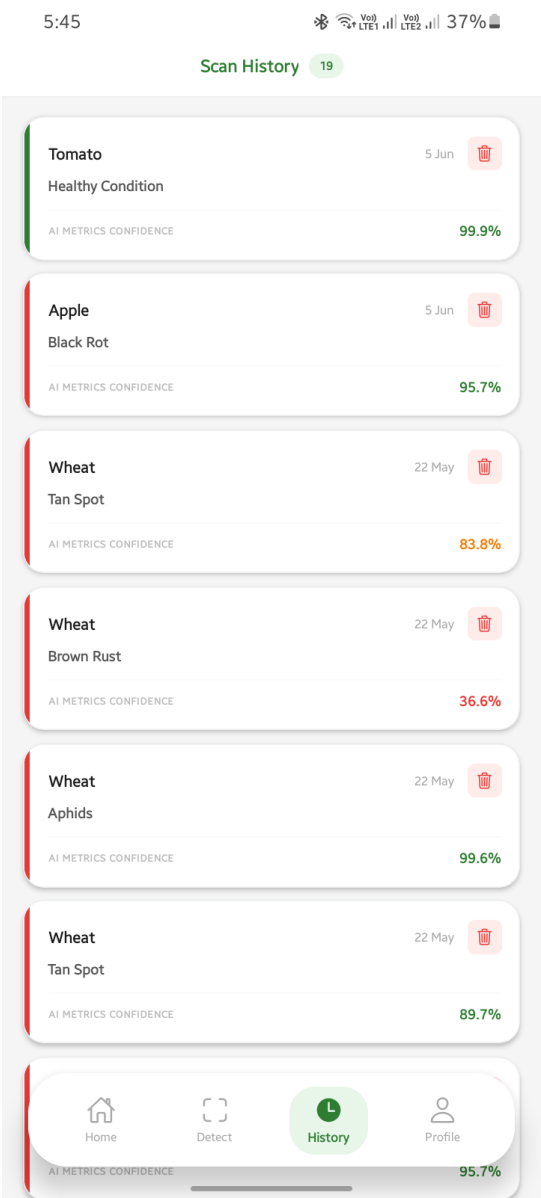


Fig. 19: History Screen

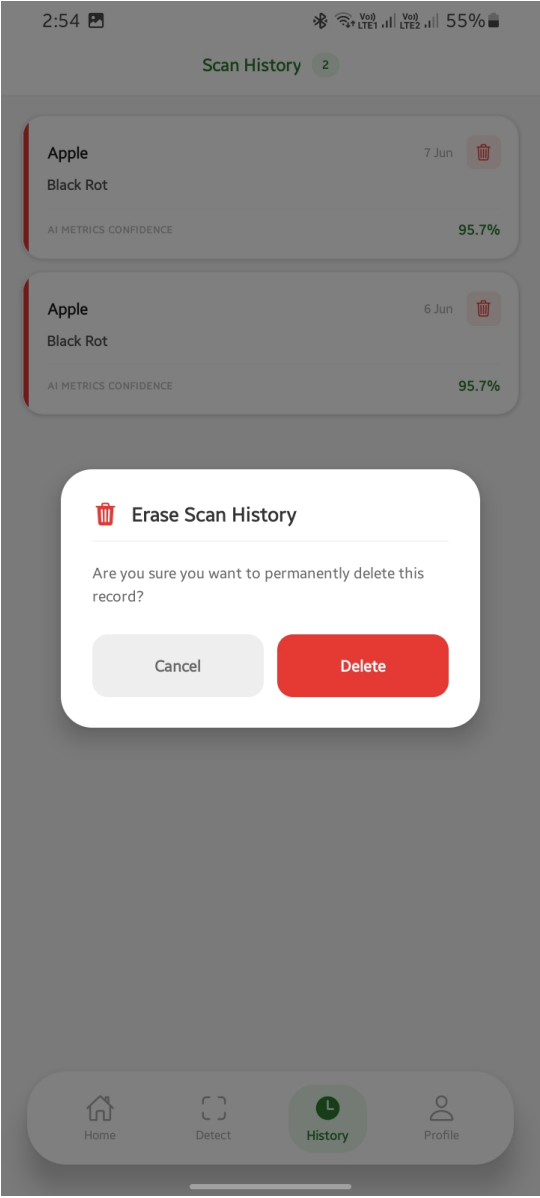


Figure 20: History Delete Confirmation

4.1.7 Water Management Screen

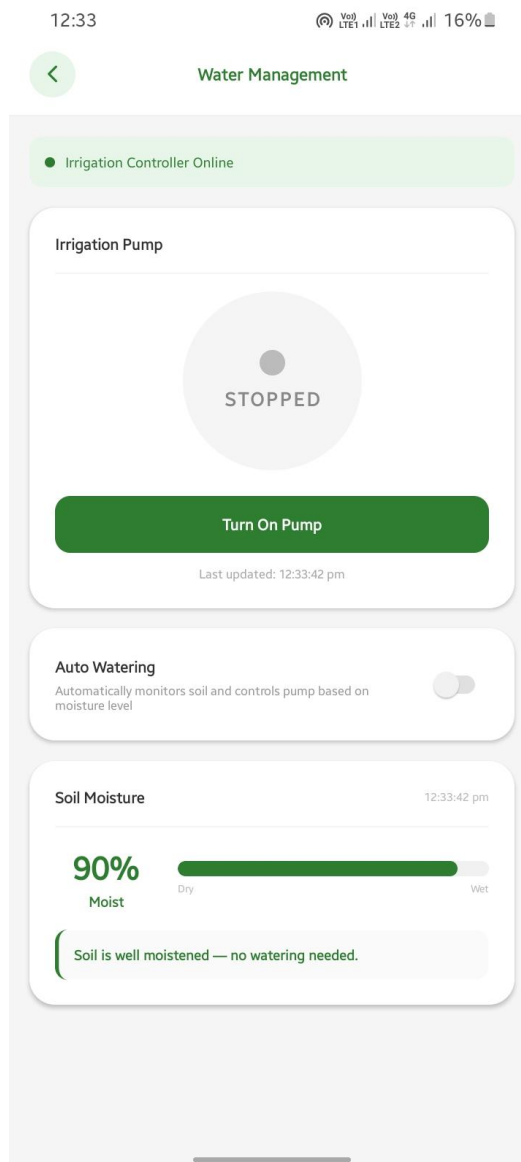


Fig. 21: Water Management Screen

4.1.8 Profile Screen and Edit Profile Screen

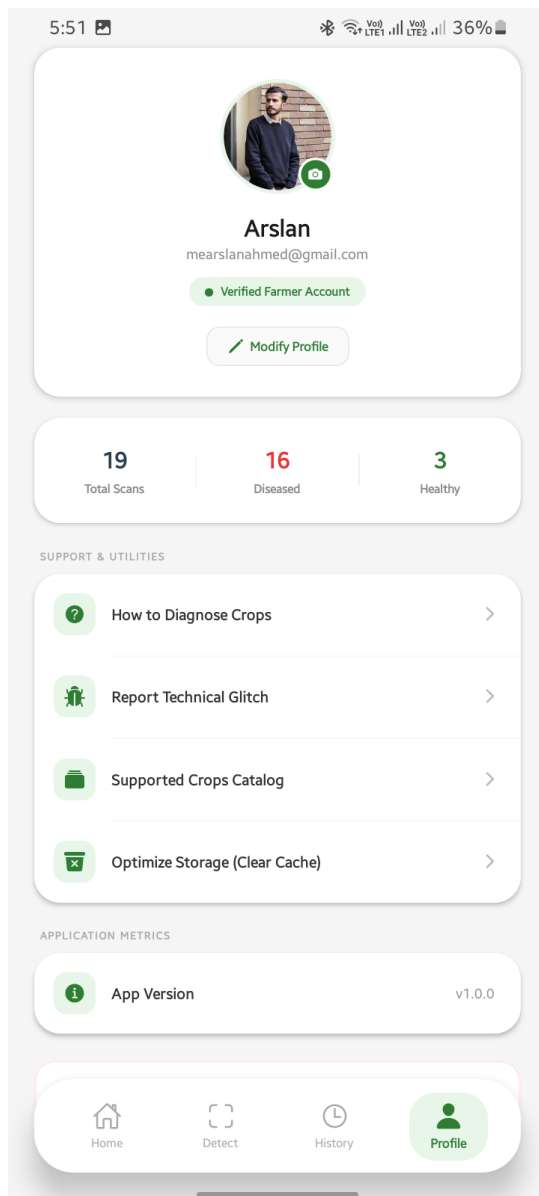


Fig. 22: Profile Screen

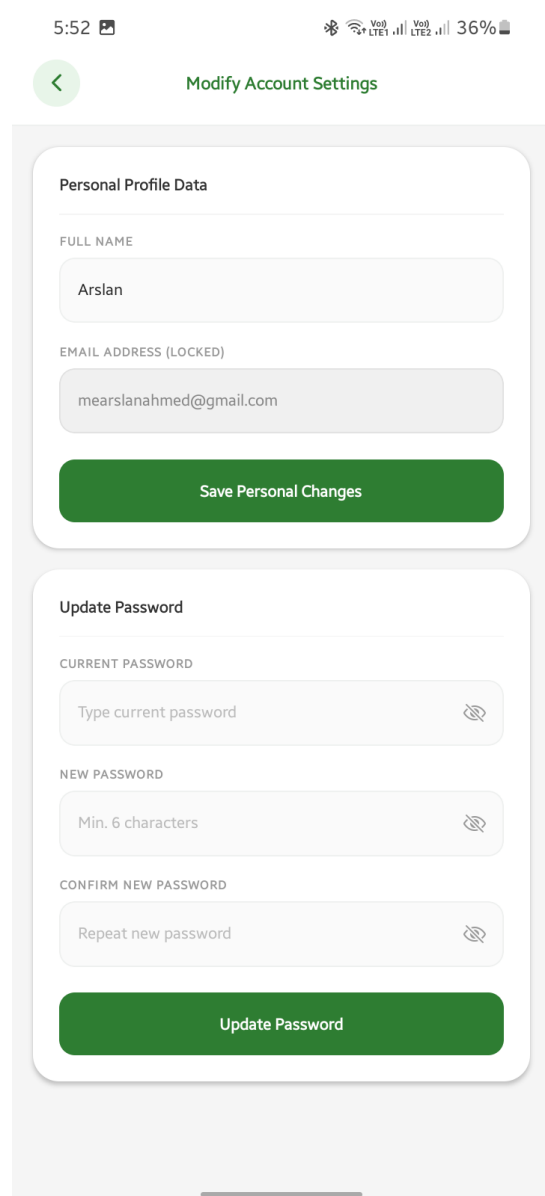


Fig. 23: Edit Profile Screen

4.1.9 Supported Crops Catalog

5. Apple 4 diseases (including healthy)
6. Corn (Maize) 4 diseases
7. Cotton 7 diseases
8. Grape 5 diseases
9. Mango 7 diseases
10. Potato 3 diseases
11. Rice 4 diseases

12. Sugarcane 8 diseases

13. Tomato 10 diseases

14. Wheat 6 diseases

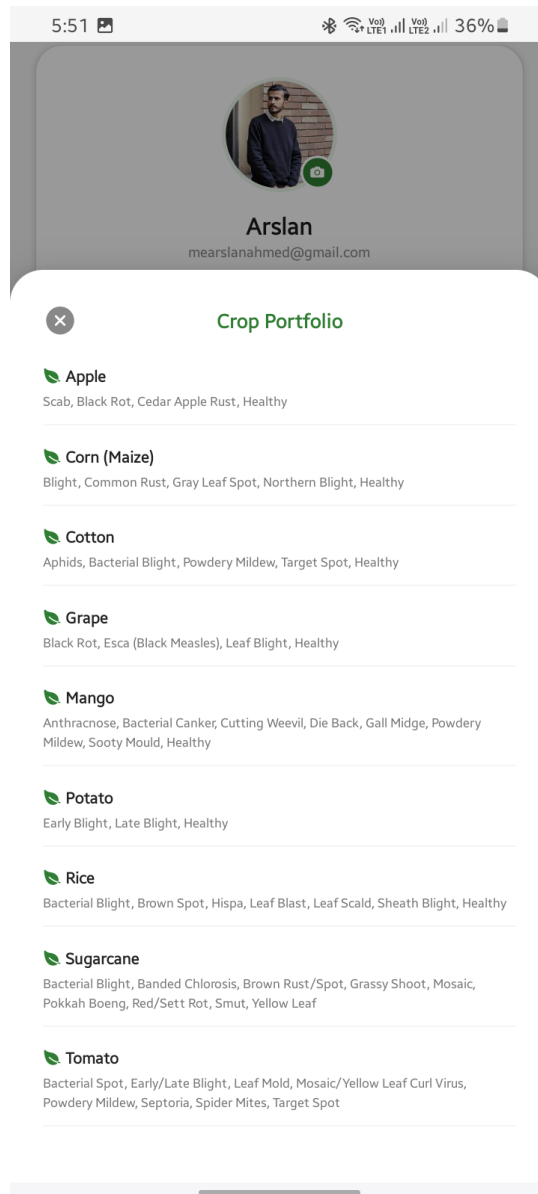


Fig. 24: Supported Crops Catalog

4.2 Backend API Implementation

The backend API gateway is built on Node.js and Express, providing routing, middleware processing, and JSON response formatting. It serves as the primary authentication server, a data persistence layer interfacing with MongoDB Atlas, and a secure reverse proxy for all ESP32 IoT device communications.

4.2.1 Server Entry Point (server.js)

```
require('dotenv').config();

const express = require('express'), cors = require('cors'), app = express();

require('./config/db')(); // Connect to MongoDB Atlas

app.use(cors());

app.use(express.json());

// Core Microservice Routes

app.use('/api/auth', require('./routes/authRoutes'));

app.use('/api/pesticides', require('./routes/pesticideRoutes'));

app.use('/api/scans', require('./routes/scanRoutes'));

app.use('/api/esp', require('./routes/espRoutes'));

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
```

4.2.2 Database Connection (config/db.js)

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`MongoDB connection error: ${error.message}`);
    process.exit(1); // kill the server if DB can't connect
  }
};

module.exports = connectDB;
```

4.2.3 Brevo Email Service (config/email.js)

```
const sendEmail = async ({ to, subject, html }) => {
  await fetch('https://api.brevo.com/v3/smtp/email', {
```

```

method: 'POST',

headers: { 'api-key': process.env.BREVO_API_KEY, 'Content-Type': 'application/json'
},

body: JSON.stringify({

  sender: { email: process.env.VERIFIED_SENDER_EMAIL, name: 'AgriGuard
Security' },

  to: [{ email: to }],

  subject,

  htmlContent: html

})

});

};

module.exports = sendEmail;

```

4.2.4 JWT Authentication Middleware (middleware/authMiddleware.js)

The authentication middleware protects routes by extracting the JWT from the Authorization: Bearer <token> header, verifying it against the JWT_SECRET environment variable, and attaching the user object (excluding the password field) to the request. If the token is missing, invalid, expired, or the user has been deleted, a 401 response is returned.

```

const jwt = require('jsonwebtoken');

const User = require('../models/User');

const protect = async (req, res, next) => {

  try {

    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) return res.status(401).json({
message: 'No token' });

    const decoded = jwt.verify(authHeader.split(' ')[1], process.env.JWT_SECRET);

```

```

    req.user = await User.findById(decoded.id).select('-password'); // Attach user to
request

    if (!req.user) return res.status(401).json({ message: 'User not found' });

    next();

  } catch (err) {

    return res.status(401).json({ message: 'Token invalid' });

  }

};

module.exports = { protect };

```

4.2.5 Authentication Controller (controllers/authController.js)

```

const jwt = require('jsonwebtoken');

const generateToken = (id) => jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn:
'30d' });

const sendVerificationOTP = async (req, res) => {

  try {

    const { email, purpose } = req.body;

    if (!email || !purpose) return res.status(400).json({ message: 'Email and purpose
required' });

    const emailLower = email.toLowerCase().trim();

    const otpCode = Math.floor(100000 + Math.random() * 900000).toString(); // Generate
6-digit OTP

    const hashedOtp = await bcrypt.hash(otpCode, 10);

    await OTP.deleteMany({ email: emailLower, purpose });

```

```
    await OTP.create({ email: emailLower, otp: hashedOtp, purpose }); // Stored with 5-
minute TTL
```

```
    // Compressed HTML email template
```

```
    const html = `

```
 await sendEmail({ to: emailLower, subject: `AgriGuard Code: ${otpCode}`, html });
```



```
 res.json({ success: true, message: 'Verification dispatched' });
```



```
 } catch (err) {
```



```
 res.status(500).json({ message: 'OTP Dispatch Failure' });
```



```
 }
```



```
};
```



```
const getMe = async (req, res) => res.json({ id: req.user._id, name: req.user.name, email:
req.user.email, role: req.user.role });
```



```
const updateProfile = async (req, res) => {
```



```
 try {
```



```
 const user = await User.findById(req.user._id);
```



```
 if (req.body.name) user.name = req.body.name;
```



```
 if (req.body.email) user.email = req.body.email;
```



```
 await user.save();
```



```
 res.json({ id: user._id, name: user.name, email: user.email });
```



```
 } catch (err) { res.status(500).json({ message: 'Profile save fail' }); }
```



```
};
```



51


```



```

const changePassword = async (req, res) => {
  try {
    const user = await User.findById(req.user._id);

    if (!(await user.matchPassword(req.body.currentPassword))) return
    res.status(401).json({ message: 'Current password match failed' });

    user.password = req.body.newPassword; // Triggers mongoose pre-save bcrypt hook
    await user.save();

    res.json({ message: 'Password change tracking success' });
  } catch (err) { res.status(500).json({ message: 'Password modification sequence dropped'
  }); }
};

```

4.2.6 Scan Controller (controllers/scanController.js)

```

const Scan = require('../models/Scan');

// POST /api/scans" Save a new scan result
const createScan = async (req, res) => {
  const { crop, disease, confidence, is_healthy, image_uri } = req.body;
  const scan = await Scan.create({
    user: req.user._id,
    crop, disease, confidence, is_healthy, image_uri,
  });
  res.status(201).json(scan);
};

// GET /api/scans" Fetch user's scans, newest first
const getScans = async (req, res) => {
  const limit = parseInt(req.query.limit) || 20;
  const scans = await Scan.find({ user: req.user._id })
    .sort({ createdAt: -1 })

```

```

    .limit(limit);
    res.json(scans);
  };

  // DELETE /api/scans/:id" Owner-only deletion with authorization check
  const deleteScan = async (req, res) => {
    const scan = await Scan.findById(req.params.id);
    if (!scan) return res.status(404).json({ message: 'Scan not found.' });

    // Security: verify the requesting user owns this scan
    if (scan.user.toString() !== req.user._id.toString()) {
      return res.status(403).json({ message: 'Not authorized.' });
    }

    await scan.deleteOne();
    res.json({ message: 'Scan deleted.' });
  };

  module.exports = { createScan, getScans, deleteScan };

```

4.2.7 Pesticide Controller (controllers/pesticideController.js)

```

const Pesticide = require('../models/Pesticide');

// GET /api/pesticides/:class_name
// Called by mobile app after detection to get advisory for detected disease
const getPesticideByClass = async (req, res) => {
  try {
    const rawParam = req.params.class_name.trim();
    // Escape regex structural characters like ( ) so they evaluate as literal text
    const escapedParam = rawParam.replace(/[-\/\^$*+?.()|[\]{}]/g, '\\$&');
    // Anchored case-insensitive regex match to eliminate string discrepancies
    const record = await Pesticide.findOne({
      class_name: { $regex: new RegExp(`^${escapedParam}$`, 'i') }
    });

    if (!record) {
      console.log(`[DATABASE 404] No pesticide record matched the key:
      "{rawParam}"`);
    }
  }

```

```

    return res.status(404).json({
      message: `No pesticide info found for: ${rawParam}`
    });
  }
  res.json(record);
} catch (err) {
  console.error('Pesticide fetch error:', err.message);
  res.status(500).json({ message: 'Server error fetching pesticide info' });
}
};

// GET /api/pesticides” Return all advisory records (for Supported Crops catalog)
const getAllPesticides = async (req, res) => {
  try {
    const records = await Pesticide.find({}).sort({ crop: 1 });
    res.json(records);
  } catch (err) {
    console.error('Pesticide list error:', err.message);
    res.status(500).json({ message: 'Server error fetching pesticide list' });
  }
};

module.exports = { getPesticideByClass, getAllPesticides };

```

4.2.8 ESP Device Controller (controllers/espController.js)

```

const EspDevice = require('../models/EspDevice');

const esp = require('../services/espService');

const capture = async (req, res) => {
  try {
    const imageBuffer = await esp.captureImage();

    res.set('Content-Type', 'image/jpeg');

    res.send(imageBuffer);
  } catch (err) { res.status(503).json({ message: 'ESP32-CAM unavailable' }); }
}

```

```

};

// Proxy Status & Relay Commands

const status = async (req, res) => { try { res.json(await esp.getStatus()); } catch (err) {
res.status(503).json({ status: 'offline' }); } };

const wroomStatus = async (req, res) => { try { res.json(await esp.getWroomStatus()); }
catch (err) { res.status(503).json({ status: 'offline' }); } };

const pumpOnHandler = async (req, res) => { try { res.json(await esp.pumpOn()); } catch
(err) { res.status(503).json({ message: 'WROOM offline' }); } };

const pumpOffHandler = async (req, res) => { try { res.json(await esp.pumpOff()); }
catch (err) { res.status(503).json({ message: 'WROOM offline' }); } };

const autoWaterHandler = async (req, res) => { try { res.json(await esp.autoWater()); }
catch (err) { res.status(503).json({ message: 'WROOM offline' }); } };

const autoOnHandler = async (req, res) => { try { res.json(await esp.autoOn()); } catch
(err) { res.status(503).json({ message: 'WROOM offline' }); } };

const autoOffHandler = async (req, res) => { try { res.json(await esp.autoOff()); } catch
(err) { res.status(503).json({ message: 'WROOM offline' }); } };

// Dynamic Hardware Registration

const registerDevice = async (req, res) => {

  try {

    const { deviceId, ip } = req.body;

    await EspDevice.findOneAndUpdate({ deviceId }, { ip, lastSeen: Date.now() }, {
upsert: true, new: true });

    res.json({ message: 'Device registered.', deviceId, ip });

  } catch (err) { res.status(500).json({ message: 'Registration failed.' }); }

};

```

```
const listDevices = async (req, res) => { try { res.json(await EspDevice.find({})); } catch
(err) { res.status(500).json({ message: 'Fail' }); } };

module.exports = { capture, status, wroomStatus, pumpOnHandler, pumpOffHandler,
autoWaterHandler, autoOnHandler, autoOffHandler, registerDevice, listDevices };
```

4.2.9 ESP Service Module (services/espService.js)

```
const axios = require('axios');

const EspDevice = require('../models/EspDevice');

// Fetch device IP dynamically from MongoDB to avoid hardcoded IPs

const getDeviceIP = async (deviceId) => {

  const device = await EspDevice.findOne({ deviceId });

  if (!device) throw new Error(`Device ${deviceId} not registered.`);

  return `http://${device.ip}`;

};

const captureImage = async () => Buffer.from((await axios.get(`${await
getDeviceIP('esp-cam')}/capture`, { responseType: 'arraybuffer', timeout: 10000 })).data);

const getStatus = async () => (await axios.get(`${await getDeviceIP('esp-cam')}/status`, {
timeout: 5000 })).data;

const getWroomStatus = async () => (await axios.get(`${await getDeviceIP('esp-
wroom')}/status`, { timeout: 5000 })).data;

const pumpOn = async () => (await axios.get(`${await getDeviceIP('esp-
wroom')}/pump/on`, { timeout: 5000 })).data;

const pumpOff = async () => (await axios.get(`${await getDeviceIP('esp-
wroom')}/pump/off`, { timeout: 5000 })).data;

const autoWater = async () => (await axios.get(`${await getDeviceIP('esp-
wroom')}/auto`, { timeout: 5000 })).data;

const autoOn = async () => (await axios.get(`${await getDeviceIP('esp-
wroom')}/auto/on`, { timeout: 5000 })).data;
```

```
const autoOff = async () => (await axios.get(`${await getDeviceIP('esp-wroom')}/auto/off`, { timeout: 5000 })).data;

module.exports = { captureImage, getStatus, getWroomStatus, pumpOn, pumpOff, autoWater, autoOn, autoOff };
```

4.3 Database Model Implementation

Data persistence is managed by MongoDB using the Mongoose ODM. The schema design encompasses collections for user profiles, encrypted credentials, historical scan results, device registration IPs, and dynamically mapped pesticide treatments.

4.3.1 User Model (models/User.js)

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Name is required'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Email is required'],
      unique: true,
      lowercase: true,
      trim: true,
    },
    password: {
      type: String,
      required: [true, 'Password is required'],
      minlength: 6,
    },
  },
```

```

    role: {
      type: String,
      enum: ['farmer', 'admin'],
      default: 'farmer',
    },
  },
  { timestamps: true }
);

```

```

// Hash password before saving (pre-save middleware)

```

```

userSchema.pre('save', async function () {
  if (!this.isModified('password')) return;
  this.password = await bcrypt.hash(this.password, 10);
});

```

```

// Method to compare login password with stored hash

```

```

userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

```

```

module.exports = mongoose.model('User', userSchema);

```

4.3.2 Scan Model (models/Scan.js)

```

const mongoose = require('mongoose');

```

```

// Stores every detection result - links back to user who scanned

```

```

const scanSchema = new mongoose.Schema(
  {
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
    crop: { type: String, required: true },
  }
);

```

```

    disease: { type: String, required: true },
    confidence: { type: Number, required: true },
    is_healthy: { type: Boolean, default: false },
    image_uri: { type: String }, // local URI from phone" for display in history
  },
  { timestamps: true } // createdAt used for history sorting
);

```

```

module.exports = mongoose.model('Scan', scanSchema);

```

4.3.3 Pesticide Model (models/Pesticide.js)

```

const mongoose = require('mongoose');

// Each document = one disease and its recommended pesticide treatment
const pesticideSchema = new mongoose.Schema(
  {
    // Must match class_name returned by Flask API exactly
    class_name: {
      type: String,
      required: true,
      unique: true,
      trim: true,
    },
    crop: { type: String, required: true, trim: true },
    disease: { type: String, required: true, trim: true },
    is_healthy: { type: Boolean, default: false },

    // Pesticide recommendation - null if plant is healthy
    pesticide: {
      name: String,      // e.g. "Copper Oxychloride"
      dosage: String,    // e.g.

```


4.3.4 OTP Model (models/OTP.js)

```
const mongoose = require('mongoose');

const otpSchema = new mongoose.Schema({
  email: {
    type: String,
    required: true,
    lowercase: true,
    trim: true
  },
  otp: {
    type: String,
    required: true
  },
  purpose: {
    type: String,
    enum: ['register', 'reset'],
    required: true
  },
  createdAt: {
    type: Date,
    default: Date.now,
    expires: 300 // Document self-destructs after 5 minutes (MongoDB TTL index)
  }
});

module.exports = mongoose.model('OTP', otpSchema);
```

4.3.5 EspDevice Model (models/EspDevice.js)

```
const mongoose = require('mongoose');

const espDeviceSchema = new mongoose.Schema({
  deviceId: { type: String, required: true, unique: true },
```

```

ip: { type: String, required: true },
lastSeen: { type: Date, default: Date.now },
});

```

```

module.exports = mongoose.model('EspDevice', espDeviceSchema);

```

4.4 ML Inference Server Implementation

The machine learning server is implemented in Python using Flask and deployed on Hugging Face Spaces. It loads two EfficientNetB3 models into memory to perform sequential crop classification and disease diagnosis.

4.4.1 Complete Flask Server (flask_api/app.py)

```

app = Flask(__name__)

app.config['MAX_CONTENT_LENGTH'] = 5 * 1024 * 1024 # 5MB limit

MODEL_DIR = os.path.join(os.path.dirname(__file__), 'model')

print("Loading ML models into memory (~260MB)...")

model1 = load_model(os.path.join(MODEL_DIR, 'AgriGuard_Model1_Final.keras')) #
Crop classifier

model2 = load_model(os.path.join(MODEL_DIR, 'AgriGuard_Model2_Final.keras')) #
Disease diagnoser

idx_to_crop = json.load(open(os.path.join(MODEL_DIR, 'model1_crop_mapping.json')))

idx_to_disease = json.load(open(os.path.join(MODEL_DIR,
'model2_disease_mapping.json')))

crop_disease_map = {}

for i, lbl in idx_to_disease.items():

    if '(' in lbl: crop_disease_map.setdefault(lbl[lbl.rfind('(')+1:-1], []).append((int(i), lbl))

def preprocess(img_bytes):

```

```

    return
np.expand_dims(np.array(Image.open(io.BytesIO(img_bytes)).convert('RGB').resize((30
0, 300))) / 255.0, axis=0)

@app.route('/predict', methods=['POST'])

def predict():

    if 'image' not in request.files or request.files['image'].filename == "": return
    jsonify({'error': 'No image'}), 400

    try: tensor = preprocess(request.files['image'].read())

    except Exception as e: return jsonify({'error': f'Invalid image: {str(e)}'}), 400

    # STAGE 1: CROP CLASSIFICATION

    crop_probs = model1.predict(tensor, verbose=0)[0]

    crop_idx, runner_up_idx = int(np.argmax(crop_probs)), int(np.argsort(crop_probs)[::-
1][1])

    crop_name, runner_up_name = idx_to_crop[str(crop_idx)],
idx_to_crop[str(runner_up_idx)]

    crop_conf = float(crop_probs[crop_idx])

    if crop_conf < 0.45 or crop_name == 'Unknown':

        return jsonify({'success': False, 'error': 'Plant unsupported or image blurry.', 'crop':
'Unknown'}), 422

    # STAGE 2: DISEASE DIAGNOSIS

    disease_probs = model2.predict(tensor, verbose=0)[0]

    candidates = crop_disease_map.get(crop_name, [])

    if candidates:

        best_idx, best_label = max(candidates, key=lambda x: disease_probs[x[0]])

    else:

        best_idx = int(np.argmax(disease_probs))

        best_label = idx_to_disease[str(best_idx)]

```

```

disease_conf = float(disease_probs[best_idx])

# DYNAMIC FALLBACK VERIFICATION

if disease_conf < 0.30 and crop_probs[runner_up_idx] > 0.20:

    runner_candidates = crop_disease_map.get(runner_up_name, [])

    if runner_candidates:

        b_run_idx, b_run_label = max(runner_candidates, key=lambda x:
disease_probs[x[0]])

        if float(disease_probs[b_run_idx]) > disease_conf:

            crop_name, crop_conf = runner_up_name, float(crop_probs[runner_up_idx])

            best_label, disease_conf = b_run_label, float(disease_probs[b_run_idx])

# FINAL GATE

if disease_conf < 0.35:

    return jsonify({'success': False, 'error': 'Crop recognized but cannot reliably diagnose
anomalies.'}), 422

    return jsonify({'success': True, 'crop': crop_name, 'disease_label': best_label,
'confidence': disease_conf})

```

4.5 ESP32 Hardware Firmware

The IoT subsystem relies on two primary microcontrollers programmed via the Arduino IDE. The ESP32-CAM acts as a remote image acquisition node, capturing and transmitting high-resolution leaf photos to the cloud.

4.5.1 ESP32-CAM Firmware (esp32_cam.ino)

```

void startCamera() {

    camera_config_t config;

    config.pixel_format = PIXFORMAT_JPEG;

    // PSRAM-adaptive settings

    if (psramFound()) {

        config.frame_size = FRAMESIZE_UXGA; // 1600x1200
    }
}

```

```

    config.jpeg_quality = 4;          // Highest quality

    config.fb_count = 2;

} else {

    config.frame_size = FRAMESIZE_SVGA; // Fallback to 800x600

}

esp_camera_init(&config);

// Sensor tuning for outdoor leaf photography

sensor_t* s = esp_camera_sensor_get();

s->set_whitebal(s, 1);

s->set_exposure_ctrl(s, 1);

s->set_gain_ctrl(s, 1);

}

void handleCapture() {

    camera_fb_t* fb = esp_camera_fb_get();

    if (!fb) return server.send(500, "text/plain", "Capture failed");

    server.send_P(200, "image/jpeg", (const char*)fb->buf, fb->len);

    esp_camera_fb_return(fb);

}

void setup() {

    WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0); // Disable brownout
detector

    startCamera();

    WiFi.setSleep(false); // Keep WiFi responsive at all times

    WiFi.begin(ssid, password);

    server.on("/capture", HTTP_GET, handleCapture);

    server.on("/status", HTTP_GET, []() { server.send(200, "application/json",
{"status":"ok"}); });

```

```

server.begin();

}

void loop() {

  server.handleClient();

}

```

4.5.2 ESP32-WROOM Pump Controller Firmware (pump_controller.ino)

```

int getMoisturePercent() {

  int raw = analogRead(MOISTURE_PIN);

  int percent = map(raw, 4095, 1400, 0, 100); // 4095=dry, 1400=wet

  return constrain(percent, 0, 100);

}

void runAutoLogic() {

  int moisture = getMoisturePercent();

  if (moisture < 30) {

    digitalWrite(RELAY_PIN, LOW); // Pump ON when soil is dry

    pumpOn = true;

  } else if (moisture > 70) {

    digitalWrite(RELAY_PIN, HIGH); // Pump OFF when soil is wet

    pumpOn = false;

  }

  // Between 30-70%: no change (hysteresis prevents rapid oscillation)

}

void registerWithBackend() {

  WiFiClientSecure client;

  client.setInsecure(); // Skip TLS cert verification for development

  HTTPClient http;

```

```

String payload = "{\"deviceId\":\"esp-wroom\",\"ip\":\"\" + WiFi.localIP().toString() +
\"\"}";

http.begin(client, backendURL);

http.addHeader("Content-Type", "application/json");

http.POST(payload); // Dynamically register IP with the Render backend

http.end();

}

void handlePumpOn() {

  digitalWrite(RELAY_PIN, LOW);

  pumpOn = true;

  server.send(200, "application/json", "{\"pump\":\"on\"}");

}

void handlePumpOff() {

  digitalWrite(RELAY_PIN, HIGH);

  pumpOn = false;

  server.send(200, "application/json", "{\"pump\":\"off\"}");

}

```

4.6 Mobile App Service Layer

The mobile service layer abstracts all external HTTP communications using Axios. It manages secure token storage, handles global application state via React Context, and provides modularized functions for user authentication, AI inference requests, and IoT hardware telemetry.

4.6.1 Central Configuration (config.js)

```

// Central config only change IP here when network changes
export const FLASK_URL = 'https://mearslanahmed-agriguard-flask.hf.space';
export const BACKEND_URL = 'https://agriguard-g3hq.onrender.com';

```

4.6.2 API Service (services/api.js)

```
import axios from 'axios';
import { BACKEND_URL } from '../config';

const api = axios.create({
  baseURL: BACKEND_URL,
  timeout: 15000, // 15 seconds" image uploads can be slow
  headers: {
    'Content-Type': 'application/json',
  },
});

// Attach JWT token to every request automatically
api.interceptors.request.use((config) => {
  // Token will be injected per-request from SecureStore when needed
  return config;
});

export default api;
```

4.6.3 Auth Service (services/authService.js)

```
import * as SecureStore from 'expo-secure-store';
import * as FileSystem from 'expo-file-system/legacy';
import * as ImageManipulator from 'expo-image-manipulator';
import { FLASK_URL, BACKEND_URL } from '../config';

export const detectDisease = async (imageUri) => {

  // Step 1: Compress image

  const compressed = await ImageManipulator.manipulateAsync(imageUri, [{ resize: {
width: 800 } }], { compress: 0.7, format: ImageManipulator.SaveFormat.JPEG });

  // Step 2: Send to Flask ML Server

  const formData = new FormData();

  formData.append('image', { uri: compressed.uri, type: 'image/jpeg', name: 'crop.jpg' });
```



```

    const mlResult = await (await fetch(`${FLASK_URL}/predict`, { method: 'POST',
body: formData })).json();

    const token = await SecureStore.getItemAsync('userToken');

    const headers = { Authorization: `Bearer ${token}` };

    // Step 3: Fetch Pesticide Advisory

    const pesticideResponse = await
fetch(`${BACKEND_URL}/api/pesticides/${encodeURIComponent(mlResult.disease_la
bel)}`, { headers });

    const pesticideData = pesticideResponse.ok ? await pesticideResponse.json() : null;

    // Step 4: Save to Database (non-blocking / fire-and-forget)

    fetch(`${BACKEND_URL}/api/scans`, {
      method: 'POST',
      headers: { ...headers, 'Content-Type': 'application/json' },
      body: JSON.stringify({ ...mlResult, image_uri: imageUri })
    });

    return { mlResult, pesticideData };
  };

export const detectDiseaseFromESP = async () => {
  const token = await SecureStore.getItemAsync('userToken');

  const tempUri = FileSystem.cacheDirectory + 'esp_capture.jpg';

  // Download raw image from ESP32-CAM via backend proxy

  const { uri } = await
FileSystem.downloadAsync(`${BACKEND_URL}/api/esp/capture`, tempUri, { headers:
{ Authorization: `Bearer ${token}` } });

  return await detectDisease(uri); // Run the detection pipeline
};

```

4.7.1 Cloud Deployment Summary

Table 16: External Service Dependencies

Service	Platform	URL
Express Backend API	Render (Node.js Web Service)	https://agriguard-g3hq.onrender.com
Flask ML API + Models	Hugging Face Spaces (Docker)	https://huggingface.co/spaces/mearslanahmed/AgriGuard-Flask
MongoDB Database	MongoDB Atlas (Cloud Cluster)	Atlas cluster with SSL/replication
Email Service	Brevo (Transactional Email API)	https://api.brevo.com/v3/smtp/email
Mobile App APK	GitHub Releases	https://github.com/mearslanahmed/AgriGuard/releases/tag/v1.0.0

4.7.2 Deployment Rationale

Render was chosen for the Express backend because it offers free-tier Node.js hosting with automatic HTTPS, easy environment variable management, and auto-deploy from GitHub. Hugging Face Spaces was selected for the Flask ML server because it provides free ML-optimized hosting with support for large model files, Docker-based deployment, and automatic scaling for inference workloads.

Chapter 5

Software Test Documentation

5 Software Test Documentation

5.1 Project Testing

Testing is a critical phase in the software development lifecycle that ensures the developed system meets its specified requirements and functions correctly under expected and edge-case conditions. For AgriGuard, testing was conducted across all four tiers of architecture: the mobile application, Express backend, Flask ML server, and ESP32 hardware firmware.

Testing was performed manually through direct interaction with the system. Each module was tested individually (unit-level) and as part of the integrated system (integration and system-level).

5.2 Purposes of Testing

The primary purpose of testing AgriGuard was to rigorously verify that every module, ranging from authentication to smart water management, operated strictly according to its specifications, while validating that the system genuinely met real-world agricultural needs. This comprehensive approach ensured early defect detection within the machine learning pipelines and hardware communication protocols, guaranteed reliability assurance by validating critical safety mechanisms like the pump failsafe and OTP expiration and assessed overall system performance to ensure rapid inference responses and graceful handling of hardware connectivity drops.

5.3 Benefits of Software Testing

5.3.1 Efficient Testing

Manual testing utilized a structured, scenario-based approach that exercised specific user workflows, such as registration, disease detection, and device control, to ensure comprehensive coverage of both success and failure paths.

5.3.2 Reusability

Test scenarios were documented as highly repeatable procedures, allowing complex tests like the full disease detection pipeline to be rapidly re-executed across all supported crops following bug fixes.

5.3.3 Consistency

Consistent procedures were universally applied, testing every API endpoint with valid inputs, missing parameters, and unauthorized access attempts to verify robust, standardized error handling.

5.3.4 Reliability

Rigorous testing focused on validating mission-critical reliability features, including the default-OFF pump failsafe on boot, precise 5-minute OTP expirations, strict ownership verification for deletions, and graceful offline device handling.

5.4 Testing Levels Applied

The following testing levels were applied to AgriGuard:

Table 17: Testing Strategy and Scope

Level	Scope	Method
Unit Testing	Individual functions and API handlers	Manual testing of each handler with various inputs
Integration Testing	Interaction between mobile app, backend, Flask server, and hardware	End-to-end workflow testing
System Testing	Complete system behavior under realistic conditions	Full scenario testing on physical devices
User Acceptance Testing	Validation with target users	Testing by team members simulating farmer workflows
ML Model Evaluation	Classification accuracy and reliability	Confusion matrices and training curves

5.5 User Acceptance Testing

User acceptance testing (UAT) validated that the system meets the practical needs of its intended user's farmers who need quick disease diagnosis and irrigation control.

UAT scenarios included:

1. A farmer registers, verifies email, and logs in successfully
2. A farmer photographs a diseased tomato leaf and receives the correct diagnosis with treatment advisory
3. A farmer checks soil moisture, manually turns the pump on, and verifies water flow
4. A farmer enables auto watering and confirms the pump activates when soil is dry
5. A farmer views scan history and deletes an old record

5.5.1 Who Performs User Acceptance Testing

UAT was performed by the development team members acting as end users, along with peer reviewers who tested the application on their own devices without prior knowledge of the system's internal workings.

5.6 Prerequisites of User Acceptance Testing

Before User Acceptance Testing (UAT) could proceed, several critical system prerequisites were fulfilled to ensure a complete testing environment. All backend API endpoints were fully deployed and functional, and both machine learning models were successfully trained, evaluated, and loaded into the active Flask server. The physical infrastructure was established by assembling and connecting the ESP32-CAM and ESP32-WROOM hardware nodes to the local network. Additionally, the backend database was completely seeded with over 76 detailed pesticide advisory records, ensuring the mobile application was fully operational and ready for installation via Expo Go on all test devices.

5.7 System Testing

System testing evaluated AgriGuard as a complete integrated system, testing the interaction between the mobile app, Express backend, Flask ML server, MongoDB database, and ESP32 hardware.

5.7.1 Method

System testing was conducted by executing end-to-end user workflows on physical Android devices connected to the same local network as the ESP32 hardware and backend servers. The Flask ML server was run locally with both models loaded into memory.

5.7.2 Tasks

Table 18: Core Functional Test Cases

Test Case	Input	Expected Result	Actual Result
Register new user	Valid name, email, password, correct OTP	Account created, JWT issued, navigate to home	Pass
Register duplicate email	Already-registered email	Error: "Email already registered"	Pass
Login with valid credentials	Correct email and password	JWT issued, navigate to home	Pass
Login with wrong password	Correct email, wrong password	Error: "Invalid credentials"	Pass
Detect healthy tomato leaf	Clear photo of healthy tomato	Crop: Tomato, Disease: Healthy, is_healthy: true	Pass
Detect diseased potato leaf	Photo showing late blight symptoms	Crop: Potato, Disease: Late Blight, confidence > 35%	Pass
Detect unsupported plant	Photo of a flower (not in training data)	Error: "Plant not supported" (422)	Pass
Detect blurry image	Very low-quality photo	Error: low confidence rejection	Pass
Fetch pesticide advisory	After successful detection of Early Blight	Advisory with dosage, spray interval, safety	Pass
Save scan to history	After successful detection	Scan appears in history screen	Pass
Delete scan	Own scan record	Scan removed from history	Pass
Delete other	Scan belonging to	Error: "Not authorized"	Pass

Test Case	Input	Expected Result	Actual Result
users' scan	different user	(403)	
Manual pump ON	ESP32-WROOM online, auto mode OFF	Pump activates, status shows ON	Pass
Manual pump OFF	Pump currently ON	Pump deactivates, status shows OFF	Pass
Enable auto mode	ESP32-WROOM online	Auto badge shows ON, manual button disabled	Pass
Auto pump activation	Auto mode ON, moisture below 30%	Pump activates automatically	Pass
Auto pump deactivation	Auto mode ON, moisture above 70%	Pump deactivates automatically	Pass
Disable auto mode	Auto mode currently ON	Auto badge OFF, pump turns OFF (failsafe), manual re-enabled	Pass
ESP32 offline handling	ESP32-WROOM powered off	Offline indicator shown, controls disabled	Pass

5.7.3 When is it Performed

System testing was performed after all individual modules were implemented and integrated, during the final phase of development (Phase 6, Weeks 16-17).

5.7.4 Who Performs It

System testing was performed by the development team, with each member testing workflows on separate devices to ensure cross-device compatibility.

5.8 Unit Testing

Key validations included ensuring the generateToken function properly encoded user IDs with strict 30-day expirations and confirming the machine learning preprocess function accurately formatted image bytes into normalized (1, 300, 300, 3) tensors. Additionally,

tests confirmed the secure generation, hashing, and delivery workflow of the sendVerificationOTP service, verified that the deleteScan handler actively prevented unauthorized deletions through ownership checks, and mathematically validated that the ESP32's raw ADC sensor values were correctly mapped to accurate 0-100% soil moisture readings.

5.8.1 Who Performs It

Unit testing was performed by the developer during implementation, testing each function immediately after writing it.

5.9 Integration Testing

Integration testing verified the communication between system components:

Table 19: System Integration Testing

Integration Point	Test	Result
Mobile App, Express Backend	JWT-authenticated API calls	Pass token correctly attached and verified
Mobile App, Flask Server	Direct multipart image upload	Pass image received, processed, result returned
Express Backend, MongoDB	CRUD operations on all 5 collections	Pass documents created, read, updated, deleted correctly
Express Backend, ESP32-WROOM	Proxied pump control and status queries	Pass commands forwarded, responses proxied back
Express Backend, ESP32-CAM	Proxied image capture	Pass JPEG binary correctly forwarded
ESP32-WROOM, Express Backend	Device self-registration on boot	Pass IP address upserted in EspDevice collection
Express Backend Brevo SMTP API	OTP email delivery	Pass emails received within seconds

5.10.1 Model 1: Crop Classifier Evaluation

Model 1 (EfficientNetB3, 11 classes) was evaluated after training using a confusion matrix and training/validation performance curves.

The confusion matrix visualizes classification accuracy across all 11 crop classes, showing correct predictions on the diagonal and misclassifications in off-diagonal cells. The evaluation graph plots training and validation accuracy and loss over training epochs, demonstrating model convergence and the absence of significant overfitting.

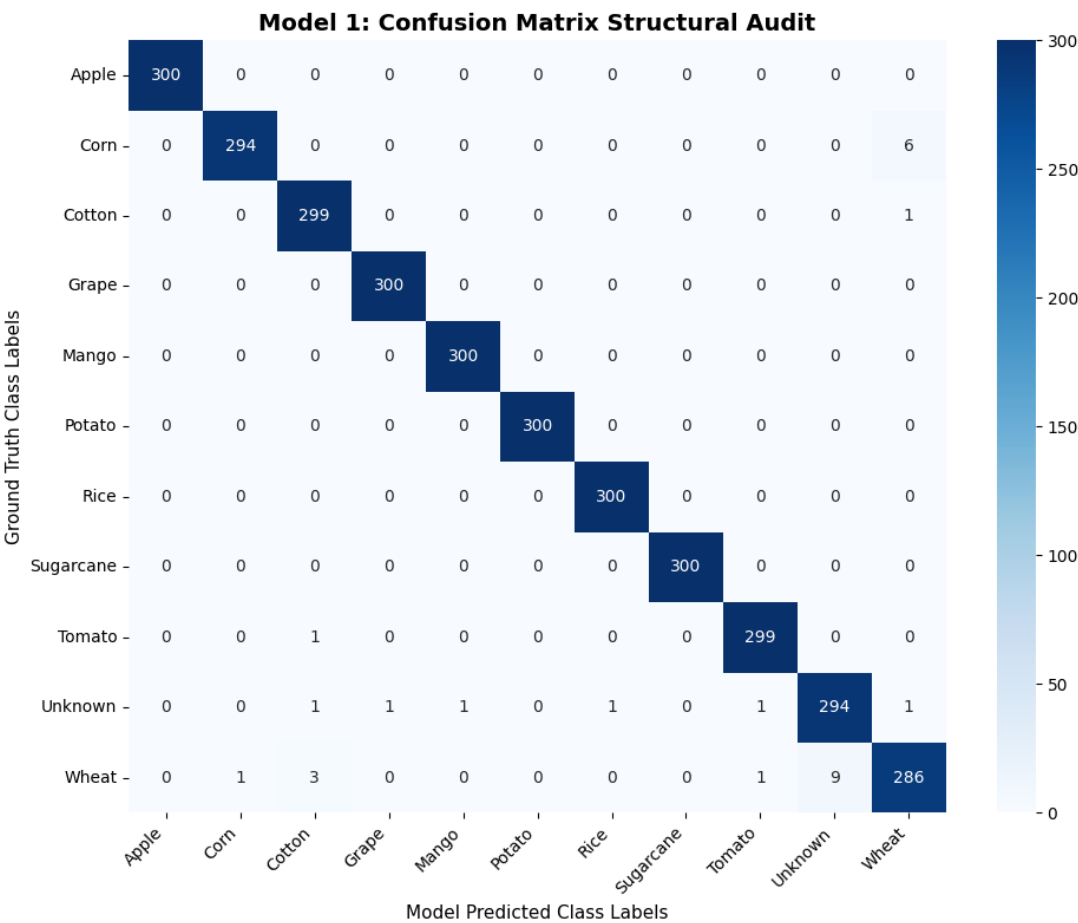


Fig. 25: Model 1 Confusion Matrix

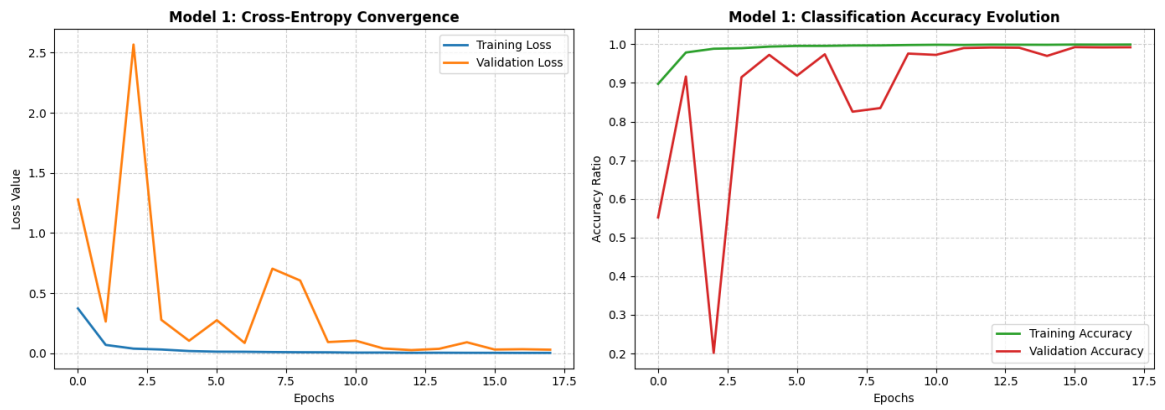


Fig. 26: Model 1 Training/Validation Curves

5.10.2 Model 2: Disease Diagnoser Evaluation

Model 2 (EfficientNetB3, 76 classes) was evaluated using the same methodology. Due to the larger number of classes, the confusion matrix is more complex, with some visually similar diseases showing higher misclassification rates (e.g., different types of blight across crops). The training/validation curves demonstrate the model's learning progression over epochs, showing the convergence of both accuracy and loss metrics.

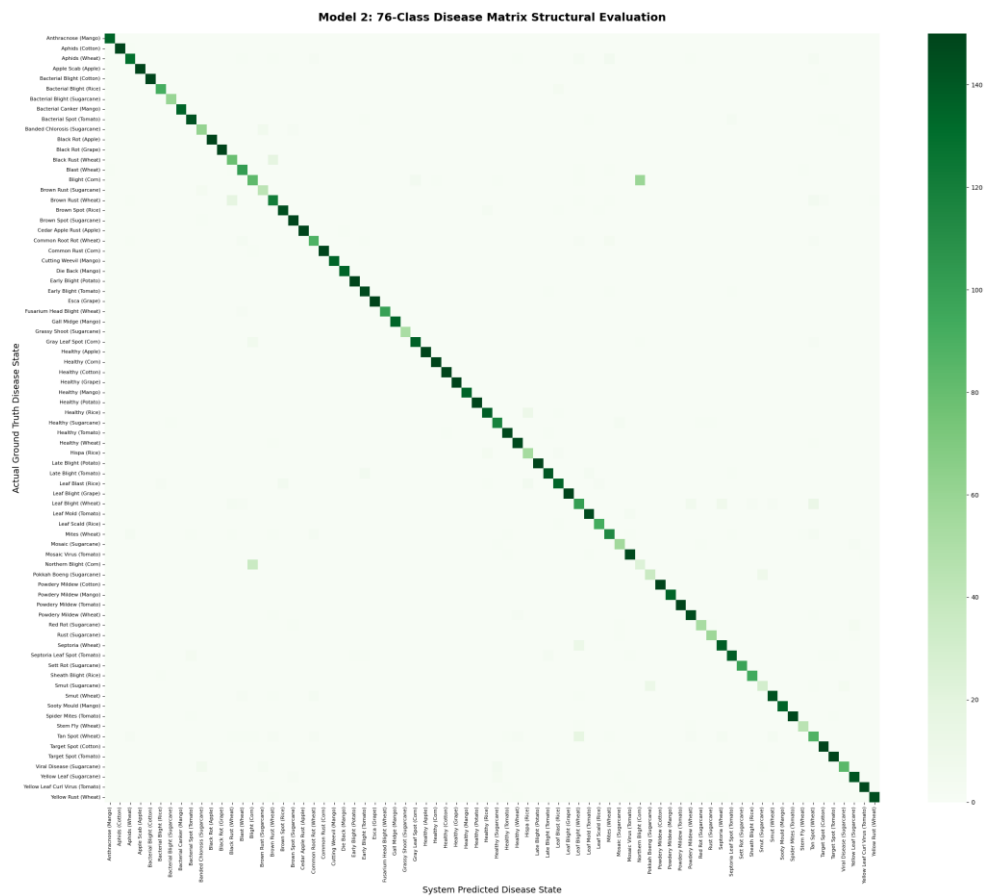


Fig. 27: Model 2 Confusion Matrix

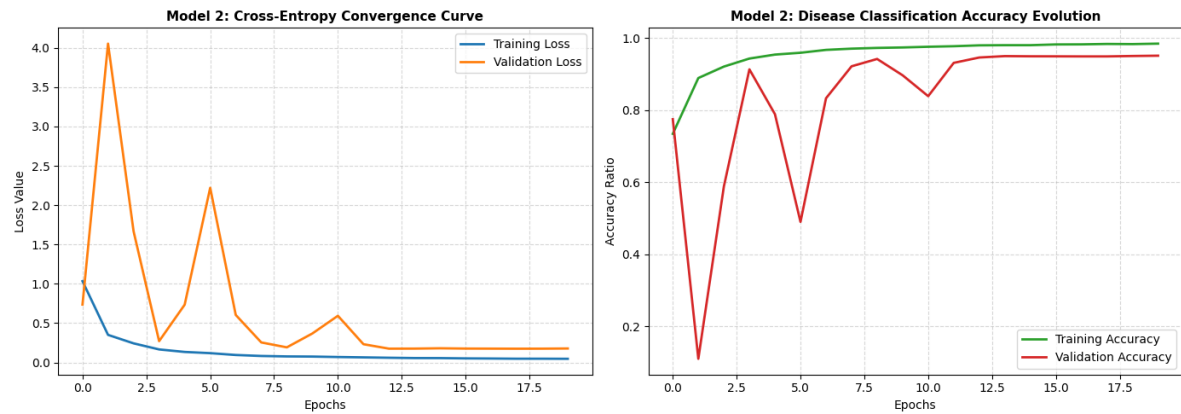


Fig. 28: Model 2 Training/Validation Curves

The confidence thresholds used in the production system (crop 0.45 (45%), disease 0.35 (35%)) were determined empirically through testing with various leaf images, balancing between accepting marginal predictions and maintaining diagnostic reliability.

5.11 Acceptance Testing

Acceptance testing confirmed that AgriGuard meets the functional and non-functional requirements defined in Chapter 2. The following acceptance criteria were verified:

Table 20: User Acceptance Criteria

Requirement	Acceptance Criterion	Status
FR-AUTH-01	Users can register with name, email, password	Accepted
FR-AUTH-02	Registration requires valid 6-digit OTP	Accepted
FR-DET-01	Users can capture leaf images with phone camera	Accepted
FR-DET-06	Crop confidence below 0.45 results in rejection	Accepted
FR-DET-10	Disease of confidence below 0.35 results in rejection	Accepted
FR-PEST-02	Advisory includes dosage, spray interval, safety info	Accepted
FR-WATER-05	Auto mode activates pump when moisture < 30%	Accepted
FR-WATER-09	Pump defaults to OFF on device boot	Accepted
NFR-SEC	Passwords are bcrypt-hashed, tokens stored in	Accepted

Requirement	Acceptance Criterion	Status
	SecureStore	
NFR-REL	OTP auto-expires after 5 minutes	Accepted

5.12.1 Registration Validation

Table 21: Validation Rules - User Input

Input	Validation Rule	Error Response
Missing name	Name field is required	400: "Name is required"
Missing email	Email field is required	400: "Email and purpose required"

5.12.2 Login Validation

Table 22: Validation Rules - Device Data

Input	Validation Rule	Error Response
Non-existent email	User.findOne returns null	401: "Invalid credentials"
Wrong password	bcrypt.compare returns false	401: "Invalid credentials"

5.12.3 Disease Detection Validation

Table 23: Validation Rules - API Requests

Input	Validation Rule	Error Response
No image file in request	'image' not in request.files check	400: "No image provided"
Empty filename	file.filename == " check	400: "Empty filename"

5.12.4 Water Management Validation

Table 24: System Behavior Validation

Scenario	Validation Rule	Behavior
Pump toggle while auto mode active	Manual button disabled in UI	User cannot trigger manual pump

Scenario	Validation Rule	Behavior
Pump ON command	Confirmation modal required	Pump activates only after confirmation
Auto mode disabled	Failsafe triggers pump OFF	Pump guaranteed OFF after auto disable
Device boot	Relay initialized HIGH	Pump starts in OFF state
Device offline	HTTP request timeout	Offline indicator shown, controls disabled

5.13 Implementation

Implementation of AgriGuard involved the systematic development and integration of all system components. The mobile app was built using React Native with Expo, the backend was deployed on Node.js with Express, the ML server was set up with Flask and TensorFlow, and the ESP32 firmware was developed using Arduino IDE.

The main steps of implementation included:

1. Setting up the development environment (Node.js, Python, Arduino IDE, Expo CLI)
2. Training and evaluating the ML models on Google Colab
3. Developing the backend API and database schemas
4. Building mobile application screens and service layer
5. Writing and uploading ESP32 firmware for both camera and pump controller
6. Integrating all components through API communication
7. Testing the complete system on physical devices and hardware

5.14 System Verification

System verification was systematically executed to ensure every component of AgriGuard produced accurate outputs for given inputs. This multi-layered process included rigorous API response testing via Postman to validate HTTP status codes and JSON payloads, alongside direct database inspection using MongoDB Compass to confirm document structures, timestamps, and relational integrity. Additionally, the machine learning inference pipeline was strictly evaluated by passing labeled images to the Flask endpoint to verify diagnostic accuracy against expected crop and disease labels.

On the physical layer, ESP32 serial monitors were utilized to confirm hardware stability, sensor accuracy, and successful backend IP registration, while the mobile application underwent extensive cross-platform testing on physical devices.

5.15 Alpha Testing

Alpha testing was conducted internally by the development team to rigorously identify and resolve critical system issues prior to external release. During this phase, several major backend optimizations were implemented: dynamic IP registration failures were resolved using MongoDB upsert operations, and machine learning accuracy was significantly stabilized by introducing strict dual-threshold dynamic fallbacks to prevent incorrect crop pivoting. Additionally, mobile processing delays on older devices were eliminated by capping image compression to 800px width at 70% quality, and unreliable OTP email delays were permanently fixed by migrating from traditional SMTP protocols to the robust Brevo REST API.

5.16 Beta Testing & Methodologies

Beta testing involved distributing the application to an external group of users to validate system performance under realistic, multi-faceted conditions. This included Focused Beta testing to evaluate machine learning robustness under varying lighting angles, Technical Beta testing across complex university WiFi networks to verify hardware communication stability, and Traditional Beta testing for comprehensive usability feedback. This external feedback directly drove several major user experience improvements, including the implementation of critical confirmation modals to prevent accidental pump activations, explicit online/offline device connection indicators, contextual moisture advisory prompts, and significantly clearer, farmer-friendly error messaging.

Chapter 6

Post Mortem Report

6 Post Mortem Report

6.1 Roles & Tasks

AgriGuard was developed as a team project with the following role distribution:

Table 25: Team Roles and Responsibilities

Team Member	Role	Responsibilities
Arslan Ahmed Naseem	Team Lead / Full Stack Developer, Testing, Documentation	React Native mobile app, ESP32 firmware, ML model training, system architecture, database design, final documentation
Amna Ikram	Developer, Documentation & Testing	Backend (Express API), Flask ML server, SRS documentation, progress reports, testing procedures, user acceptance testing

6.2 What Worked Well

Several critical technical aspects of the project progressed exceptionally smoothly, validating the underlying system architecture. The dual-stage machine learning pipeline proved highly effective, successfully reducing false positive rates while its dynamic fallback mechanism seamlessly handled borderline crop identifications without manual intervention.

On the hardware side, the dynamic IP registration protocol elegantly solved the challenge of unpredictable local networks, eliminating the need for complex static IP configurations. Development velocity was further accelerated by the Expo Managed Workflow, which provided robust pre-built modules without requiring native linking, alongside MongoDB's TTL indexing, which automatically purged expired OTPs to reduce backend codebase complexity. Finally, implementing the Express backend as a

secure reverse proxy successfully centralized all hardware management and securely masked local device IP addresses from the mobile client.

6.3 Team Effectiveness

The development team operated highly effectively through consistent communication and a clear division of technical responsibilities. This effectiveness was primarily driven by regular progress meetings with the project supervisor, providing clear direction and strict accountability throughout the project lifecycle. To facilitate parallel development without codebase conflicts, the team utilized Git for centralized version control. Furthermore, adopting agile iterative development cycles allowed the team to proactively identify and resolve complex hardware-software integration issues exceptionally early, while shared cross-role testing responsibilities ensured the final system was rigorously validated from both developer and end-user perspectives.

6.4 Lessons Learned

Key lessons include the importance of decoupling the heavy AI inference server from the primary Node.js backend to prevent blocking, optimizing image compression on the mobile client before upload to save bandwidth, and utilizing hardware interrupts and watchdogs to stabilize ESP32 WiFi connections.

1. **Hardware Integration Requires Early Testing:** Early testing across diverse WiFi networks is critical, as complex hardware anomalies, such as the ESP32-CAM's brownout detector triggering unexpected resets, took significant time to properly diagnose.
2. **ML Threshold Tuning is Empirical:** Strict diagnostic confidence thresholds (45% for crops, 35% for diseases) cannot be set theoretically; they require extensive empirical testing across varied image datasets to perfectly balance valid predictions against unreliable anomalies.
3. **Client-Side Compression Matters:** Aggressively implementing client-side image compression (800px width at 70% quality) was essential to prevent severe network timeouts when uploading full-resolution photos over slow mobile data.
4. **Separate ML Server Architecture Pays Off:** Decoupling the heavy AI inference engine into an independent Python Flask server drastically simplified deployment and

utilized native TensorFlow support, keeping the primary Express backend incredibly lightweight.

5. **Database Seeding Needs Verification:** Bulk pesticide seeder logic required meticulous string validation, as even minor formatting mismatches between the ML model's output and database class names caused difficult-to-debug advisory errors.

6.5 Conclusion of the Project

AgriGuard successfully demonstrates the integration of advanced deep learning with affordable IoT hardware to solve critical agricultural challenges. By automating disease diagnosis and irrigation, the platform provides a scalable, accessible tool for modernizing crop management and improving yield outcomes.

1. **AI-Powered Disease Detection:** A mobile application that enables farmers to photograph crop leaves and receive instant disease diagnosis through a dual-stage EfficientNetB3 pipeline supporting 10 crops and 76 disease conditions.
2. **Actionable Treatment Advisories:** Disease-specific pesticide recommendations with dosage, spray intervals, water ratios, and safety precautions, sourced from a database of 76+ verified advisory records.
3. **IoT Smart Irrigation:** A hardware system consisting of ESP32 microcontrollers with soil moisture sensing and relay-controlled pump operation, supporting both manual remote control and automatic threshold-based watering. The ESP32-WROOM registers dynamically with the Render-hosted backend, enabling seamless communication through a reverse proxy pattern.
4. **Secure Authentication:** Email-based OTP verification with bcrypt hashing, JWT-based session management with 30-day expiry, and platform-native secure token storage.
5. **Complete Record Keeping:** A scan history system that preserves all diagnostic results with user-ownership authorization, enabling farmers to track crop health patterns over time.